Cryptnox ID CLI

Release 1.1.0

Cryptnox

Nov 12, 2026

GENERAL

1 Overview **3**
- 1.1 Reachability at a glance . . . 3
- 1.2 Readers . . . 3
- 1.3 Platforms . . . 4

2 Installation **5**
- 2.1 Install . . . 5
- 2.2 PC/SC per platform . . . 5
- 2.3 Windows single-file executable . . . 6
- 2.4 WSL2 (Windows Subsystem for Linux) . . . 6
- 2.5 Next . . . 6

3 Getting started **7**
- 3.1 Choosing a reader . . . 7
- 3.2 Global options . . . 8
- 3.3 One card operation at a time . . . 8
- 3.4 Development keys . . . 8
- 3.5 Next . . . 9

4 Interactive shell **11**
- 4.1 Built-ins and keys . . . 11
- 4.2 Global options carry in . . . 11
- 4.3 Desktop shortcut (Windows) . . . 12
- 4.4 Piped input . . . 12

5 Command syntax & conventions **13**
- 5.1 Global options . . . 13
- 5.2 Top-level commands . . . 14

6 JSON output **15**
- 6.1 The contract . . . 15
- 6.2 Result envelope . . . 15
- 6.3 Error object . . . 15
- 6.4 Snapshot reports . . . 16
- 6.5 Remote service operations . . . 17

7 Exit codes **19**

8 Troubleshooting **21**
- 8.1 Reader and transport . . . 21
- 8.2 Status words and errors . . . 21

8.3 FIDO2 . . . 22
8.4 Genuineness . . . 22
8.5 Remote service . . . 23
8.6 yubico-piv-tool interop . . . 23
8.7 Getting a shareable diagnostic . . . 23

9 Factory & provisioning **25**
9.1 Pre-personalization profiles . . . 25
9.2 Factory commands . . . 28

10 License **31**

11 Quick start: a working PIV card **39**
11.1 Prerequisites . . . 39
11.2 The one-command path . . . 39
11.3 The steps, by hand . . . 39
11.4 Verify with yubico-piv-tool . . . 41
11.5 Interoperability matrix . . . 41
11.6 If something fails . . . 42
11.7 Next steps . . . 42

12 Windows logon & Remote Desktop **43**
12.1 How it has to look . . . 43
12.2 Step 0 — pre-personalize with the ms-logon profile (once per card) . . . 44
12.3 Step 1 — get the certificate issued . . . 44
12.4 Step 2 — verify Windows sees the card . . . 46
12.5 Step 3 — log on . . . 46
12.6 Scope and limits . . . 46
12.7 Next steps . . . 46

13 SSH public-key authentication **47**
13.1 How it has to look . . . 47
13.2 Why it fails on cryptnox-default . . . 47
13.3 Fixing it: give 9A the SIGN role . . . 48
13.4 Prerequisites . . . 49
13.5 Step 0 — find the PKCS#11 module path . . . 49
13.6 Step 1 — get the SSH public key off the card . . . 49
13.7 Step 2 — authorize it . . . 50
13.8 Step 3 — load it into ssh-agent . . . 50
13.9 Step 4 — connect . . . 50
13.10 Troubleshooting . . . 50
13.11 Next steps . . . 51

14 SSH user certificates **53**
14.1 Prerequisites . . . 53
14.2 Step 1 — export the card's public key . . . 53
14.3 Step 2 — the CA signs it . . . 53
14.4 Step 3 — trust the CA on the server . . . 54
14.5 Step 4 — connect with the certificate . . . 54
14.6 Troubleshooting . . . 54
14.7 Next steps . . . 55

15 TLS client authentication **57**
15.1 How it has to look . . . 57
15.2 Prerequisites . . . 57

15.3 Firefox (NSS, cross-platform) . . . 58
15.4 Windows / IIS (native store) . . . 58
15.5 curl / OpenSSL (scripted clients) . . . 59
15.6 Troubleshooting . . . 60
15.7 Next steps . . . 60

16 macOS smart-card login **61**
16.1 How it has to look . . . 61
16.2 Cards on the cryptnox-default profile . . . 61
16.3 Prerequisites . . . 62
16.4 Step 0 — fix macOS's CCID driver . . . 62
16.5 Step 1 — pair the certificate to the account . . . 63
16.6 Step 2 — use it . . . 63
16.7 Requiring the card (optional, higher risk) . . . 63
16.8 Troubleshooting . . . 63
16.9 Next steps . . . 64

17 PIV personalization **65**
17.1 Quickstart (one shot) . . . 65
17.2 Step 0 — factory: pre-personalize the applet structure . . . 66
17.3 Step 1 — set the PIN and PUK . . . 66
17.4 Step 2 — generate keys on the card . . . 66
17.5 Step 2b — import an external private key . . . 67
17.6 Step 3 — CSR and certificates (on-card signing) . . . 67
17.7 Step 4 — data objects . . . 68
17.8 Step 5 — validate and smoke-test . . . 68
17.9 Step 6 — finalize (irreversible, factory) . . . 68
17.10 Rotating the PIV admin keys . . . 68
17.11 Next steps . . . 70

18 PIV commands **71**
18.1 Inspection . . . 71
18.2 PIN . . . 71
18.3 PUK . . . 71
18.4 Admin channel . . . 72
18.5 Certificates and objects . . . 73
18.6 Attestation . . . 73
18.7 Personalization (`piv perso`) . . . 73
18.8 Notes . . . 74

19 Quick start: a passkey on the card **75**
19.1 Credential management . . . 75
19.2 authenticatorConfig policy . . . 76
19.3 Reset . . . 76

20 FIDO2 guide **77**
20.1 Windows: run elevated . . . 77
20.2 Read-only commands . . . 77
20.3 Write operations . . . 77
20.4 Credential management . . . 78
20.5 authenticatorConfig policy . . . 78
20.6 Scope . . . 79
20.7 Next steps . . . 79

21 FIDO2 commands **81**

21.1 Inspection . . . 81
21.2 PIN . . . 81
21.3 Credentials . . . 81
21.4 Configuration . . . 81
21.5 Destructive . . . 82

22 Quick start: a tamper-evident NFC tag (SDM/SUN) 83
22.1 Prerequisites . . . 83
22.2 Step 1 — create an application . . . 83
22.3 Step 2 — set up the SDM file . . . 83
22.4 Step 3 — read and verify . . . 84
22.5 Scope and production notes . . . 84
22.6 Next steps . . . 84

23 MIFARE DESFire guide 85
23.1 Inspect (read-only) . . . 85
23.2 Applications and files . . . 85
23.3 Authenticate, write, read . . . 85
23.4 Delete (authenticated) . . . 86
23.5 Value files . . . 86
23.6 Record files . . . 86
23.7 Keys . . . 86
23.8 Secure Dynamic Messaging (SDM / SUN) . . . 87
23.9 Format the PICC (destructive) . . . 87
23.10 Scope . . . 87

24 MIFARE DESFire commands 89
24.1 Inspection . . . 89
24.2 Applications . . . 89
24.3 Keys . . . 89
24.4 Files and data . . . 89
24.5 Value and record files . . . 89
24.6 Secure Dynamic Messaging (EV3) . . . 90
24.7 Destructive . . . 90

25 Quick start: prove a card is genuine 91
25.1 Next steps . . . 91

26 Genuineness guide 93
26.1 What is checked — and what deliberately is not . . . 93
26.2 Trust anchors . . . 93
26.3 Development cards . . . 94
26.4 Inspection without the proof . . . 94
26.5 Next steps . . . 94

27 Genuineness commands 95
27.1 Inspection . . . 95
27.2 Verification . . . 95

28 Remote service guide 97
28.1 How a relayed operation works . . . 97
28.2 The relay policy . . . 97
28.3 Operations . . . 97
28.4 What a failed authentication costs . . . 98
28.5 The `--default-keys` flag . . . 98

28.6 What is sent to the service . . . 99
28.7 Transcripts . . . 99

29 Remote service commands **101**
29.1 Group options . . . 101
29.2 Identification and inspection . . . 101
29.3 Key attestation . . . 101
29.4 Reset . . . 102
29.5 Endpoint . . . 103

Command-line management for the Cryptnox multi-applet smart card: PIV (SP 800-73 identity credentials), FIDO2/CTAP 2.1 (passkeys), and MIFARE DESFire contactless — three independent functions on one physical card, driven by one tool: `cryptnox-id`.

CHAPTER

ONE

OVERVIEW

The Cryptnox ID card carries three independent functions on one chip:

- **PIV** — SP 800-73 identity credentials: PIN-protected keys and X.509 certificates for signing, authentication and encryption, usable by standard PIV tooling (yubico-piv-tool, OpenSC, OS smart-card stacks).
- **FIDO2 / CTAP 2.1** — passkeys over NFC for WebAuthn sign-in.
- **MIFARE DESFire** — contactless applications: access control, closed-loop counters, and, on cards with DESFire EV3, Secure Dynamic Messaging (self-authenticating NFC tags).

Cryptnox ID CLI (the `cryptnox-id` command) manages all three from one terminal.

A fourth applet, **genuineness**, is Cryptnox's factory attestation: it proves the card is authentic. Unlike the three functions above it is *inspected*, never provisioned — see *Genuineness guide*.

1.1 Reachability at a glance

Function	Interface	Notes
PIV	contact; contactless only for what needs no PIN	The PIN, the PUK and everything behind the PIN are contact-only, so Windows logon needs a contact reader; administration works over both interfaces. Over contactless the built-in profiles expose the CHUID, the card-authentication certificate and the card-authentication key (9E), which needs no PIN: the data physical access control uses.
FIDO2	contactless (NFC)	Windows: requires an Administrator terminal.
DESFire	contactless (NFC)	Requires a DESFire-capable reader (ACS ACR1252 verified).
Genuineness	contact	Cryptnox attestation applet; read-only (*Genuineness guide*).

The **same physical card** exposes **different applets** depending on the interface it is reached over. If PIV looks present but MIFARE does not, you are on a contact reader — and vice versa.

1.2 Readers

The Cryptnox card readers are the recommended choice and cover every function of the card. Any PC/SC reader also works for PIV and FIDO2; MIFARE DESFire additionally needs a reader that passes *native* DESFire APDUs — not all contactless readers do.

Reader	Use	Status
Cryptnox Smartcard Reader (contact)	PIV, FIDO2, genuineness	Cryptnox — recommended
Compact USB Mini Smartcard Reader (contact)	PIV, FIDO2, genuineness	Cryptnox — recommended
Cryptnox NFC Contactless Reader (NFC)	DESFire, PIV/FIDO2 over NFC	Cryptnox — recommended
ACS ACR39U (contact)	PIV, FIDO2, genuineness	verified
ACS ACR1252 (contactless)	DESFire, PIV/FIDO2 over NFC	verified (native DESFire OK)
Feitian R502 (contactless)	DESFire, PIV/FIDO2 over NFC	observed working
HID OMNIKEY 5422CL	PIV / GlobalPlatform only	**DESFire not supported** (frames unanswered)

Reader names differ across units, operating systems and drivers, so the CLI selects by name substring as well as index — see *Getting started*. A Cryptnox reader may report a Cryptnox-branded name ("CryptnoxCR" contact, "Cryptnox NFC" contactless) or the ACS model name of the underlying unit; run `readers` to see yours — the no-`--reader` default auto-selects both families.

1.3 Platforms

The CLI is pure Python over PC/SC and runs on Windows, Linux and macOS. All three card functions are verified on real hardware on each. The one platform-specific behaviour is **FIDO2 on Windows**, which requires an Administrator terminal (Windows reserves the CTAP interface for its WebAuthn API); Linux and macOS have no such restriction. See *Installation* for per-OS setup.

CHAPTER

TWO

INSTALLATION

`cryptnox-id` is a Python CLI (Python 3.10+) over PC/SC.

2.1 Install

Install with pipx — it puts `cryptnox-id` on your `PATH` **globally** while keeping the tool's dependencies (`click`, `rich`, `cbor2`, `cryptography`, `pyscard`, …) in their own isolated environment. It is also the method that works out of the box on current Linux distributions, where installing into the system Python is blocked (PEP 668).

```
$ pipx install cryptnox-id-cli
$ cryptnox-id --version
```

(pipx itself: `apt install pipx` / `dnf install pipx` / `brew install pipx`, or on Windows `py -m pip install --user pipx` then `py -m pipx ensurepath`.)

Windows alternative — plain pip works there and gives a global command too, as long as your Python `Scripts` directory is on `PATH`:

```
pip install cryptnox-id-cli
cryptnox-id --version
```

A classic virtual environment also works, but the command is then available only while that venv is activated — fine for development (see the README), not what you want for day-to-day card management.

Three console entry points are installed and interchangeable: `cryptnox-id`, the short `cnx-id`, and `cryptnox-id-card`.

2.2 PC/SC per platform

- **Windows** — built in (the *Smart Card* service, `SCardSvr`); nothing to install.
- **Linux** — install `pcscd` and the CCID driver (`apt install pcscd libccid`). Building `pyscard` from source also needs `build-essential swig libpcsclite-dev` (and `python3-dev` on a distro Python without headers).
- **macOS** — built in. Note two things: the system Python is 3.9 (below the 3.10 floor) — install 3.10+ with uv, Homebrew or python.org; and `cbor2` must be `<6` (enforced in packaging), since 6.x has no Intel-mac wheel.

Confirm the install sees a reader:

```
$ cryptnox-id readers
```

2.3 Windows single-file executable

A standalone `cryptnox-id.exe` (no Python install required) is built with PyInstaller from `packaging/cryptnox-id.spec`. PyInstaller cannot cross-compile, so each platform's binary is built on that platform; the wheel above is the cross-platform path and runs anywhere Python 3.10+ and PC/SC exist.

2.4 WSL2 (Windows Subsystem for Linux)

WSL2 has no PC/SC by default, but a USB reader can be passed through with usbipd-win. Verified end to end: PIV over contact, DESFire EV2 over contactless, and FIDO2 over contactless (no elevation) all work from Ubuntu in WSL2 against physical cards.

```
# Windows (Administrator), one-time per reader (find <id> with: usbipd list):
usbipd bind   --busid <id>
# per session (WSL must be running):
usbipd attach --wsl --busid <id>     # detaches the reader from Windows
```

```
# WSL (Ubuntu):
$ sudo apt install pcscd libccid pcsc-tools usbutils
$ sudo pcscd
$ cryptnox-id --reader PICC mifare info    # contactless; --reader ICC/ACR39U for contact
```

```
usbipd detach --busid <id>           # return the reader to Windows
```

Reader **names and indices differ on Linux**, so select by name substring (`PICC` = contactless, `ICC` / `ACR39U` = contact), not index. While a reader is attached to WSL, Windows cannot see it.

2.5 Next

- *Getting started* — first commands and reader selection
- *Overview* — what the card and CLI do

CHAPTER

THREE

GETTING STARTED

First contact with the card:

```
$ cryptnox-id readers       # which reader to use (note the --reader hint)
$ cryptnox-id info          # all three functions on one screen
$ cryptnox-id doctor        # if anything looks off
```

`info` and `doctor` are read-only and safe to run any time. `doctor` exits with code **2** if a blocking check fails (useful in scripts); FIDO/DESFire warnings on a contact or non-elevated setup are **not** blocking.

Then pick your path: *Quick start: a working PIV card*, *Quick start: a tamper-evident NFC tag (SDM/SUN)* or *Quick start: a passkey on the card*.

3.1 Choosing a reader

Every command talks to exactly one reader.

- **Default (no ``–reader``)** — the tool picks the single **Cryptnox or ACS** reader that has a card. This is deliberate: it avoids touching another project's card in some other reader.
- If there is **no Cryptnox/ACS reader**, or **several** readers with cards, the tool **refuses to guess** and asks you to pass `--reader`.
- **``–reader <index>``** — the number from `cryptnox-id readers` (e.g. `--reader 0`).
- **``–reader <name>``** — a case-insensitive substring of the reader name (e.g. `--reader CryptnoxCR`, `--reader ACR39`, `--reader PICC`); an ambiguous substring is an error.

Two things to know:

- Virtual readers such as **"Windows Hello for Business"** show up in `readers` — ignore them; they are not Cryptnox cards.
- The **same physical card** over a contact vs a contactless reader exposes **different applets**. If PIV looks present but MIFARE does not, you are on a contact reader — and vice versa. Reader names also differ per unit and OS/driver (`PICC` = contactless, `ICC` / `ACR39U` = contact; some Cryptnox units report `CryptnoxCR` / `Cryptnox NFC` instead), so prefer selecting by name substring.

A bench with both a contact and a contactless reader loaded:

```
$ cryptnox-id readers
#  0 | ACS ACR39U ICC Reader 0 (ACS) | present | 3B...     <- contact
#  1 | Feitian R502 0                | present | 3B...     <- contactless
$ cryptnox-id --reader 0 piv info        # PIV over contact
$ cryptnox-id --reader 1 mifare info     # DESFire over contactless
```

3.2 Global options

These sit before the command and apply to it:

Option	Meaning
`--reader <name|index>`	Select the PC/SC reader (default: the ACS reader with a card).
`--json`	Machine-readable JSON to stdout; diagnostics go to stderr.
`--verbose`	Human debug output; a redacted APDU trace to stderr.
`--apdu-log <file>`	Append a redacted APDU transcript to a file.
`--dry-run`	Write nothing to the card. Commands that can plan (`piv quickstart`, `apdu`, `factory`, the key imports) show the intended actions; every other command refuses to run under the flag rather than executing for real (fail-closed).
`--yes`	Skip destructive confirmation prompts.
`--timeout <seconds>`	Reader/card timeout.
`--no-color`	Disable coloured output.

3.3 One card operation at a time

Each `cryptnox-id` invocation opens the reader, does one thing, and releases it. Admin and personalization commands each open their **own** SCP03 session (this card accepts one application APDU per applet-directed session), so there is nothing to "keep open" between commands. For a running prompt that issues many commands in a row without re-typing the prefix, use *the interactive shell* (`cryptnox-id shell`).

3.4 Development keys

Warning

Cards ship with well-known **default** keys so that the first personalization works out of the box, and the examples in this documentation use them too:

- **PIV admin channel (SCP03)** — key version 1 of the card's PIV security domain (`A0000001515350 4101`) holds the published GlobalPlatform test key `40 41 42 ... 4F` (`--default-keys`). The card manager (ISD) is a separate domain with per-card keys that the CLI never uses;
- **PIV management key (9B)** — the same published GlobalPlatform test value, doubled for an AES-256 key object (`--default-keys`);
- **DESFire** application keys after `app create` — all-zero AES (`--zero-key`);
- **PIV PIN / PUK** in the examples — `123456` / `12345678`.

These are **not secrets** — they are public, published values. A production deployment must replace them. The environment variables only *authenticate* with keys that are already on the card; putting new admin keys on the card is a separate step, see *Rotating the PIV admin keys*. After it, the admin channel takes `PIV_SCP03_ENC` / `PIV_SCP03_MAC` / `PIV_SCP03_DEK`; `PIV_MGMT_KEY` carries the PIV management key, `CRYPTNOX_PIV_PIN` / `CRYPTNOX_PIV_NEW_PIN` / `CRYPTNOX_PIV_NEW_PUK` the PINs and PUKs, and DESFire keys use the `--key-env NAME` form.

3.5 Next

- *Quick start: a working PIV card* — provision a PIV credential end to end
- *Quick start: a passkey on the card* — FIDO2 / CTAP2 walkthrough
- *Quick start: a tamper-evident NFC tag (SDM/SUN)* — DESFire EV3 Secure Dynamic Messaging
- *Troubleshooting* — when something looks off

CHAPTER

FOUR

INTERACTIVE SHELL

`shell` starts a captive prompt that runs `cryptnox-id` subcommands without the program-name prefix — type `piv info`, not `cryptnox-id piv info` — and loops until you leave. It is built for desktop use: point a shortcut at it and card management opens as a small console, no terminal habits required.

```
$ cryptnox-id shell
cryptnox-id interactive shell. Type a command without the 'cryptnox-id' prefix (e.g.
↪'piv info').
  help            show available commands
  <command> -h    help for a command
  clear           clear the screen
  exit / quit     leave the shell

cryptnox-id> readers
...
cryptnox-id> piv info
...
cryptnox-id> exit
```

Only this CLI's own commands run — it is **not** an operating-system shell; `ls` or `dir` are unknown commands here.

4.1 Built-ins and keys

Input	Effect
`help`	List the available commands (same as `--help`).
`<command> -h`	Help for one command, e.g. `piv -h`.
`clear`	Clear the screen.
`exit` / `quit`	Leave the shell.
`Ctrl-C`	Cancel the current line or the running command; the shell stays open.
`Ctrl-D` (end of input)	Leave the shell.

Arguments follow shell quoting rules (`--subject "CN=Test User"` works); an unbalanced quote is reported and the prompt returns. Command history and line editing are available where Python's `readline` exists (Linux, macOS).

4.2 Global options carry in

Global options passed when *launching* the shell apply to every command run inside it — `--reader`, `--json`, `--verbose`, `--apdu-log`, `--dry-run`, `--yes`, `--timeout`, `--no-color`:

```
$ cryptnox-id --reader 1 --json shell
```

Every command in that session now targets reader 1 and prints JSON.

Each line is its own invocation, with a fresh card session per command — exactly as if run from the OS prompt (the SCP03 admin channel is per-command). A failing command prints its error and returns to the prompt; it never ends the shell.

4.3 Desktop shortcut (Windows)

Create a shortcut whose target is the installed binary with the `shell` argument, e.g.:

```
C:\...\Scripts\cryptnox-id.exe shell
```

Double-clicking it opens the captive prompt directly. `fido ...` commands need Administrator rights on Windows, but the shortcut does not have to be elevated: when it isn't, the CLI offers to re-run the command as Administrator — approve the UAC prompt and the result is shown back in your window (see *Quick start: a passkey on the card*). Starting the shortcut elevated once avoids the per-command prompts.

4.4 Piped input

Without a terminal the shell reads commands line by line from standard input, so short sequences can be scripted:

```
$ printf 'readers\npiv info\n' | cryptnox-id shell
```

CHAPTER

FIVE

COMMAND SYNTAX & CONVENTIONS

`cryptnox-id` (aliases: `cnx-id`, `cryptnox-id-card`) manages the three independent functions of the Cryptnox multi-applet smart card. Every command supports `-h`/`--help`.

5.1 Global options

Option	Meaning
`--reader <name|index>`	Select the PC/SC reader by substring or index (default: first ACS reader). When several candidate readers hold a card, the CLI refuses to guess — pass this explicitly.
`--json`	Machine-readable JSON on stdout; notes and warnings go to stderr. See *JSON output*.
`--verbose`	Human debug output; a redacted APDU trace on stderr.
`--apdu-log <file>`	Append a redacted APDU transcript to a file.
`--dry-run`	Write nothing to the card. Planning-capable commands (`piv quickstart`, `apdu send/select/transcript`, `factory piv preperso load-config/finalize`, `piv perso import-key/import-p12`) show the intended actions; read-only commands run normally; every other command refuses to run under the flag (fail-closed) instead of silently executing.
`--yes`	Skip confirmation prompts (use deliberately).
`--timeout <seconds>`	Reader/card timeout.
`--no-color`	Disable coloured output.

Reader requirements per function: PIV works over contact (the PIN, the PUK and the PIN-protected keys are contact-only) or contactless (administration and PIN-less reads). **MIFARE DESFire needs a DESFire-capable contactless reader. FIDO2 needs an Administrator terminal on Windows** (the OS reserves the CTAP AID otherwise).

5.2 Top-level commands

Command	Description
`readers`	List PC/SC readers, card presence and ATR; recommends a `--reader` value.
`info`	Detect the card and summarize all three functions on one screen.
`doctor`	Diagnostics: PC/SC service, reader, transport, per-function reachability, platform notes.
`apdu send`	Developer raw-APDU access (transcripts are redacted; sensitive command data is never logged).
`apdu select`	SELECT an application by AID.
`apdu transcript`	Show the redacted APDU transcript of the session.
`report card` / `report piv` / `report mifare` / `report fido` / `report genuine` / `report full`	Secret-safe JSON reports (`--out FILE`).
`remote authenticate` / `remote inspect` / `remote attest` / `remote reset` / `remote dev-reset`	Operations run through the Cryptnox remote PIV service, with the card in the local reader.
`shell`	Interactive prompt: run subcommands without the `cryptnox-id` prefix (type `piv info`); global options passed when launching carry into every command. `help` / `clear` / `exit` — see *Interactive shell*.

CHAPTER

SIX

JSON OUTPUT

Every command accepts the global `--json` flag for machine consumption.

6.1 The contract

- The **result payload** is printed to **stdout** as a single JSON document.
- Notes, warnings and progress go to **stderr** — never merge the two streams into one parser (no `2>&1`).
- Errors produce a JSON **error object** and a non-zero exit code (see *Exit codes*).
- Secrets never appear in any output stream — every transcript and log line passes a redaction layer that masks PINs, keys and other registered secrets.

`--json` is a **global** option: it goes before the command (`cryptnox-id --json fido info`), not after it.

6.2 Result envelope

On success, each command writes one JSON object whose fields are specific to that command. For example, `cryptnox-id --json doctor`:

```
{
  "checks": [
    {"check": "PC/SC service", "status": "ok", "detail": "1 reader(s) detected"},
    {"check": "Card present", "status": "ok", "detail": "ATR 3BFA1300..."}
  ],
  "ok": true
}
```

Read-only inspection commands (`info`, `piv info`/`status`, `fido info`, `mifare info` …) each return the same fields their human output shows, serialized as an object. Write commands return a small confirmation object (what was done, and any public identifier produced — never a secret).

6.3 Error object

On failure the payload is an error object and the process exits non-zero:

```
{
  "error": "card_access_denied",
  "message": "FIDO2 access was blocked by Windows. ..."
}
```

`error` is a stable token (see *Exit codes* for the token/exit-code taxonomy); `message` is human-readable. Errors that carry an ISO status word add three fields:

```
{
  "error": "status_word_error",
  "message": "...",
  "sw": "6A82",
  "sw_name": "FILE_NOT_FOUND",
  "context": "SELECT PIV"
}
```

6.4 Snapshot reports

`report card` / `report piv` / `report mifare` / `report fido` / `report genuine` / `report full` emit **secret-safe** JSON snapshots of the detected state, suitable for fleet inventory or support tickets (`--out FILE` writes to a file). These are the primary machine-consumption shapes and are kept stable.

Every report is wrapped in a common envelope:

```
{
  "generated_by": "cryptnox-id 0.1.0",
  "generated_at": "2026-08-18T12:00:00+00:00",
  "safe_to_share": true,
  "<section>": {}
}
```

`report full` carries all sections at once; the single-function reports carry just theirs. The section shapes:

card

```
{"reader": "<reader name>", "state": {}, "cplc": "<hex or null>"}
```

piv

```
{
  "state": "PivPersonalized",
  "apt": {"aid": "...", "label": "OpenFIPS201", "url": "..."},
  "pin": {"ref": "80", "configured": true, "blocked": false, "verified": false,
          "retries_remaining": 6, "refused_on_interface": false},
  "puk": {"ref": "81", "configured": true, "blocked": false, "verified": false,
          "retries_remaining": 6, "refused_on_interface": false},
  "objects_present": ["chuid", "ccc", "auth-cert"],
  "notes": []
}
```

`apt` / `pin` / `puk` are `null` when not available. `refused_on_interface` is `true` when the card answered the PIN or PUK probe with 6982, which a contactless reader gets because the built-in profiles make both contact-only; `configured` and `retries_remaining` are then unknown, not false. The same `pin` / `puk` objects appear in `info --json` and `piv status --json`; `piv pin status --json` adds a top-level `note` string when either is refused.

mifare — one of:

```
{"state": "DesfireReachable", "version": {}, "free_memory": 1234,
 "applications": ["CC0102"]}
```

```
{"state": "DesfireNeedsContactlessReader",
 "note": "DESFire ... use a DESFire-capable contactless PC/SC reader."}
```

```
{"state": "DesfireNoAnswerContactless",
 "note": "DESFire did not answer on this contactless interface. Re-present the card ..."}
```

`DesfireNeedsContactlessReader` means the session is on a contact interface, which cannot reach DESFire at all; `DesfireNoAnswerContactless` means the interface is already contactless (reader name / ATR evidence) and the card simply did not answer — re-present the card, and check the reader passes native DESFire APDUs.

fido — one of:

```
{"state": "FidoPersonalized", "select_version": "U2F_V2", "get_info": {}}
```

```
{"state": "FidoBlockedByOS", "note": "<Windows elevation guidance>"}
```

```
{"state": "FidoNotPresent"}
```

genuine — one of:

```
{"state": "GenuinenessPersonalized", "leaf_subject": "...",
 "info": "<hex>", "note": "state only; run `genuine verify` to prove ..."}
```

```
{"state": "GenuinenessNotPresent",
 "note": "genuineness applet not found (contact-only; absent over contactless)."}
```

A section may also report `{"state": "Unknown", "error": "<message>"}` when a probe fails. Nothing in any report contains a PIN, key, or private material — `safe_to_share` is always `true`.

6.5 Remote service operations

`remote` commands return one object with a fixed top level: `op`, `endpoint`, `card` (reader, ATR, CPLC UID and serial, all read locally), `result` (the service's own result object, passed through unchanged), `verified_locally` (each cross-check as `true`, `false` or `null` when the service reported nothing to compare) and `relay` (commands relayed, seconds, proof-of-work bits, the service's log lines, and `open_initialize_updates`: per security domain, the INITIALIZE UPDATEs the operation left without a completed authentication). `reset` and `dev-reset` add `before` and `after` inventories; `attest` adds `verification` with every binding checked on the returned certificate. Fields under `result` are the service's assertions; only `verified_locally`, `before`, `after` and `verification` are the tool's own findings.

CHAPTER

SEVEN

EXIT CODES

`cryptnox-id` exits non-zero on any failure; the code identifies the failure class. With `--json`, errors are also emitted as a JSON object carrying the matching machine token (see *JSON output*).

Code	Token	Meaning
0	—	Success.
1	`error` (and specific tokens such as `key_import`)	Generic CLI error not covered by a more specific class.
2	— / `desfire_frame_too_large`	Command-line usage error (unknown option, bad parameter). Also used by one DESFire transport-limit error.
3	`no_readers` / `reader_not_found` / `no_card` / `secret_input`	No usable reader/card, or a required secret could not be obtained safely (e.g. no TTY for a prompt and no environment variable).
4	`card_access_denied`	The OS refused card access — on Windows this is the FIDO applet without an Administrator terminal.
5	`applet_not_found` / `desfire_not_selected`	The requested function is not reachable on this card/interface.
6	`status_word`	The card rejected a command (the message carries the ISO status word).
7	`scp03_error` / `mgmt_key`	SCP03 secure-channel failure (wrong admin keys, cryptogram mismatch), or PIV management-key (`9B`) authentication failed.
8	`profile_error`	Pre-personalization profile parse/validation failure.
9	`desfire_error`	A DESFire command returned an error status.
10	`ctap_error`	A FIDO2/CTAP command returned an error status.
11	`ev2_error`	DESFire secure-messaging (session/MAC) failure.
12	`remote_error`	A remote PIV service operation failed for a reason without a more specific class.
13	`remote_connect`	The remote PIV service could not be reached, or the TLS handshake failed.
14	`remote_protocol`	The remote PIV service sent something the client cannot interpret safely.
15	`remote_policy`	The remote PIV service asked for a card command the relay policy refuses to send. The message names the command header.
16	`remote_failed`	The exchange completed, but the remote PIV service reported the operation as failed.

CHAPTER

EIGHT

TROUBLESHOOTING

First stop, always:

```
$ cryptnox-id doctor
```

It checks the PC/SC service, readers, transport, and each card function, and says what to fix.

Quick hits:

- **FIDO commands fail on Windows** — run from an Administrator terminal.
- **DESFire not answering** — use a DESFire-capable *contactless* reader.
- **A PIN step fails over NFC** — the PIN, the PUK and the PIN-protected keys are contact-only; administration works over both interfaces.
- **yubico-piv-tool sees nothing** — pass your reader: `-r <substring>`.
- **Two readers hold a card** — the CLI refuses to guess; pass `--reader`.

8.1 Reader and transport

"No PC/SC readers found" / cannot reach the PC/SC service. Windows: ensure the *Smart Card* service (`SCardSvr`) is running. Linux: start `pcscd` and install `libccid`. macOS: PC/SC is built in.

Multiple readers, wrong one picked. The CLI prefers ACS readers. Pass `--reader <index>` or a name substring (`--reader "ACR1252 1S CL Reader PICC"`). `cryptnox-id readers` shows the indices.

Reader compatibility (DESFire). DESFire native commands are passed through correctly by the **ACS ACR1252** (PICC interface). The **HID OMNIKEY 5422CL is incompatible** — DESFire frames go unanswered (and the link can wedge into `SCARD_E_READER_UNAVAILABLE`); PIV and GlobalPlatform still work on it. Use a DESFire-capable reader for `mifare` commands.

8.2 Status words and errors

``SCARD_E_NO_ACCESS 0x80100027`` on ``fido`` commands. Windows blocks non-elevated PC/SC access to the FIDO CTAP AID. Run the command from an **Administrator terminal**; the CLI prints exactly this guidance and can relaunch itself elevated.

``6D00`` to DESFire commands (even contactless). A JavaCard applet (e.g. PIV) is currently selected and answering the interface. DESFire is the card's **default applet**: re-present the card or open a fresh connection without selecting anything, on a DESFire-capable reader. The CLI's detector probes DESFire first for this reason.

``6700`` (wrong length) on large PIV writes. A full certificate does not fit one short APDU, and extended length is rejected on these transports. The CLI sends large objects via **ISO command chaining** (CLA `0x10`); `import-cert` does this automatically.

``6700`` on ``SELECT PIV``, only over a contactless reader (``info`` shows PIV as ``Unknown``). The multi-applet card rejects the *first* case-4 ISO SELECT (data **+ Le**) issued right after a DESFire native-command session, and the composite detector (`info` / `doctor` / `report full`) probes DESFire first. The identical SELECT sent as **case-3** (no Le) is accepted and returns the FCI, so the CLI retries once without Le. Standalone `piv ...` commands are never affected. If you script raw `apdu` calls, drop the trailing Le on the SELECT that follows DESFire native commands.

``6E27``-style failure on the second admin command in one SCP03 session. This JCOP 4.5 card accepts only **one application APDU per applet-directed SCP03 session**. The CLI opens a fresh SCP03 channel per admin/perso command (and uses ISO command chaining when one logical command must span blocks), so you should not hit this in normal use — only when scripting raw `apdu` calls.

``PIN verification refused (SW=6982)`` on the first command that asks for the PIN, and ``piv status`` / ``piv pin status`` say the PIN is not usable over this interface, right after ``set-pin`` succeeded. You are on a contactless (NFC/PICC) reader. The PIN and PUK verifiers are **contact-only** on every built-in profile, and so are the 9A/9C/9D keys; only the admin channel (`load-config`, `set-pin`, `set-puk`, `generate-key`, `import-cert`, `write-standard-objects`) works over both interfaces, which is why pre-personalization and personalization got that far. (One exception: on-card RSA generation that needs the management-key fallback, since 9B is contact-only too.) The PIN you set is on the card. Move it to a contact reader and continue with the step that failed. Over NFC the card offers the PIN-less functions only: the CHUID, the card-authentication certificate and the 9E card-authentication key.

``6985`` (conditions not satisfied) on ``generate-csr`` / signing. The slot's key has the wrong role. Signing needs a key whose role includes **SIGN**, such as **9C** (or a 9A provisioned with the `ssh` / `ms-logon` profile); a default-profile 9A is AUTHENTICATE-only and refuses `GENERAL AUTHENTICATE` signing. The error names the key's role, and `piv inventory` lists the role of every key object. A role cannot be changed; to give the slot a signing key, add a key object for another algorithm with `piv perso create-key-object` (see *PIV commands*). See *Pre-personalization profiles*.

``6986`` on ``piv inventory``. The applet's `restrict-enumeration` config flag is set, so the structure listing needs the admin role. Version, state and config are still readable.

``0xAE AUTHENTICATION_ERROR`` on ``mifare app delete``. Deleting an application requires authentication. Pass `--zero-key` (factory default) or `--key-env NAME`; the CLI then authenticates and sends a MACed DeleteApplication.

``0x9D PERMISSION_DENIED`` on ``mifare write``. The file's write access condition needs a key you have not authenticated with, or you authenticated with the wrong key number. Check `--key-no` and the file's access rights.

8.3 FIDO2

Nothing happens / access denied even as Administrator. Confirm the terminal is truly elevated (`fido info` prints a non-elevated warning otherwise). Some security software also brokers WebAuthn; close other FIDO clients.

8.4 Genuineness

``genuine`` reports "applet not found" on a contactless reader. The genuineness applet is **contact-only** — it does not answer over the contactless interface (SELECT returns `6A82`). Use a contact reader (e.g. ACS ACR39U). `info` shows `GenuinenessNeedsContactReader` when DESFire is reachable, i.e. you are on a contactless interface.

``genuine verify``: proof of possession ``valid`` but chain ``not verified`` (``NOT proven``). No Cryptnox genuineness root is pinned, so the chain cannot be anchored — the tool reports `NOT proven` rather than silently passing. Pass `--anchors DIR` with the trust anchor, or bundle the pinned root. A **dev** card chains to a throwaway dev root: `genuine verify --anchors <dev-ca-dir>`.

Genuineness is not the ISD / PIV-SSD check. `genuine verify` proves the attestation applet's device key and certificate chain. It does **not** verify the per-card ISD or PIV-SSD keys (KDF/HSM-derived) — this tool has no access to those secrets, and the output says so. The per-card PIV-SSD key set is key version 2 of the PIV security domain; see *Rotating the PIV admin keys* for what it is and why to leave it in place.

8.5 Remote service

`remote` commands exit with a code that names the failure class (see *Exit codes*):

- **13** `remote_connect`: the service could not be reached or the TLS handshake failed. Check the network; the endpoint is fixed to `wss://piv.cryptnox.com`. With `CRYPTNOX_REMOTE_URL` set, only `authenticate` and `inspect` run, and only to `wss://` or loopback.
- **14** `remote_protocol`: the service sent a frame the tool cannot interpret. Update the tool; if the newest version fails the same way, report it with the message.
- **15** `remote_policy`: the service asked for a card command the relay policy refuses; the message names the command header. Nothing was sent to the card for that command and the operation stopped. This is the tool protecting the card; report the header so the policy can be reviewed.
- **16** `remote_failed`: the exchange completed and the service reported the operation as failed; the error object carries the service's result. A per-card rate limit exists; wait before retrying. A `dev-reset` refused because the card cannot prove it is genuine, or a `reset --default-keys` refused for lack of a factory token, left the card untouched: contact Cryptnox support to have the card re-provisioned.

`remote reset` and `remote dev-reset` refuse to run non-interactively without `--i-understand-this-is-irreversible`, and `--yes` is not accepted for them. `remote attest` refuses non-interactively without `--yes` when the slot or the attestation container already holds a certificate.

8.6 yubico-piv-tool interop

``yubico-piv-tool`` sees nothing / "failed to connect". It defaults to Yubico's own reader-name matching. Pass your reader explicitly with `-r <substring>`. Note that `yubico-piv-tool` gates its connect on the applet answering a Yubico firmware-version query — OpenSC and the Windows minidriver use standard PIV discovery instead and are unaffected. See the interop matrix in *Quick start: a working PIV card*.

8.7 Getting a shareable diagnostic

```
$ cryptnox-id report full --out report.json
```

Writes a secret-safe JSON snapshot of all three functions to attach to a bug report. It never contains PINs or keys.

CHAPTER

NINE

FACTORY & PROVISIONING

Manufacturing-stage material: pre-personalization lays down the PIV applet's *structure* — data containers, PIN/PUK verifiers, key objects — from a profile, over the SCP03 admin channel. Operators normally never need this; it matters for development/evaluation cards and for defining a deployment's card profile.

9.1 Pre-personalization profiles

A **profile** describes the PIV applet structure to lay down during factory pre-personalization: the data containers, the PIN/PUK verifiers and their policy, and the asymmetric key slots with their access rules and mechanisms. `factory piv preperso load-config` turns a profile into a sequence of OpenFIPS201 `PUT DATA ADMIN` commands over SCP03. See *PIV personalization* for where this fits in the full lifecycle.

9.1.1 Built-in profiles

`cryptnox-default`
: Byte-exact to the applet's own reference profile: an AES-256 admin key, ten containers (CHUID/CCC/certs/security-object/fingerprints/facial/printed), and five ECC-P256 key slots (9A/9C/9D/9E, plus the AES-only admin key 9B). Its 9A key can authenticate but not sign, and macOS login, SSH, TLS client authentication and Windows logon all sign with 9A. `load-config` and `piv quickstart --include-preperso` warn about this for any profile whose 9A cannot sign; for those uses pick `ssh` or `ms-logon`.

`developer` / `npivp-lab`
: The same structure as `cryptnox-default` — only the `mode` label differs (`developer-not-for-production` / `lab`), for cards that should never be mistaken for production units.

`ssh`
: SSH auth (raw public key or SSH user certificates) via PKCS#11 (`ssh-agent` / OpenSC) — see *SSH public-key authentication*. Only change from `cryptnox-default`: the 9A (PIV Authentication) key gets `SIGN` added to its role (`SIGN+AUTHENTICATE`). 9C cannot be used for this regardless of role: OpenSC's PIV driver requires re-authentication before every signature on key reference 0x9C specifically (PKCS#11 `CKA_ALWAYS_AUTHENTICATE`, matching the PIV spec's non-repudiation convention for that slot), which neither `ssh-agent` nor `ssh`'s own PKCS#11 client can satisfy.

`ms-logon`
: Windows smart-card logon / Remote Desktop. This applet dispatches PKI challenge signing on `ROLE_SIGN` only, so the 9A (PIV Authentication) key gets `SIGN+AUTHENTICATE` — that's what makes client authentication (PKINIT, SSH, TLS) *and* on-card CSR generation work on 9A. A second RSA-2048 (CRT) key object coexists on 9A — same slot, different mechanism, since the applet keys objects by `(ref, mechanism)`. The role deliberately does **not** include `KEY_ESTABLISH`: the applet routes challenge-response to key transport before signing, and that role would divert it. Use `ssh` above instead if you only need SSH and don't need the extra RSA object.

That RSA object is why this profile works on Windows. The Windows inbox PIV minidriver enumerates RSA keys only; given an EC key it builds no key container at all, so the card looks empty to Windows (see *Windows logon & Remote Desktop*). The object can be filled either way:

- **generated on-card** — `piv quickstart --profile ms-logon --slot 9A --algorithm RSA2048` (the private key never leaves the card);
- **imported**, for an AD-issued credential (CSR -> CA -> import, or PKCS#12 import via *PIV personalization*).

The object carries the `IMPORTABLE` attribute so the second path is available; that attribute permits import, it does not prevent on-card generation.

9.1.2 Inspecting profiles

```
$ cryptnox-id factory piv preperso inspect-defaults
$ cryptnox-id factory piv preperso init-config --profile ms-logon --out profile.yaml
```

`inspect-defaults` shows the applet's supported algorithms, key slots, and PIN/PUK limits — no card needed. `init-config` writes a built-in profile to an editable YAML file. To capture what a *card* currently exposes (a read-only observed snapshot, not a loadable profile), use `export-config --out snapshot.yaml`.

9.1.3 Profile YAML

```yaml
name: cryptnox-default
mode: production
admin:
  key_ref: "9B"
  mechanism: AES256
pin:
  min: 6
  max: 8
  retries: 6
  charset: numeric
puk:
  min: 8
  max: 8
  retries: 6
  charset: numeric
containers:
  - oid: 5FC102
    name: chuid
    contact: ALWAYS
    contactless: ALWAYS
  - oid: 5FC109
    name: printed
    contact: PIN
    contactless: SM_PIN
keys:
  - ref: "9C"
    name: sign
    mechanism: ECCP256
    role: SIGN
    contact: PIN
```

(continues on next page)

(continued from previous page)

```
    contactless: NEVER
    attributes: [IMPORTABLE]
```

Mechanisms: `AES128`, `AES192`, `AES256`, `RSA2048`, `RSA3072`, `RSA4096`, `ECCP256`, `ECCP384`, `CS2`, `CS7`.

Access modes (used for `contact`/`contactless` on both containers and keys):

- `ALWAYS`: no condition.
- `NEVER`: not accessible over that interface.
- `PIN`: after the PIN has been verified.
- `OCC`: after an on-card biometric comparison.
- `SM`: only inside a PIV secure-messaging session (SP 800-73-4).
- `SM_PIN`: `SM` plus the PIN.
- `VCI`: `SM`, and the session also meets the virtual contact interface condition (SP 800-73-4).
- `VCI_PIN`: `VCI` plus the PIN.

Profile files written by version 1.1 use `VCI` and `VCI_PIN` for what is now `SM` and `SM_PIN`; `load-config` warns when a file uses either name.

Neither this tool nor the Windows inbox minidriver opens PIV secure-messaging sessions, so anything marked `SM`, `SM_PIN`, `VCI` or `VCI_PIN` on an interface cannot be used there with them. In the built-in profiles that is the contactless side of most containers. The PIN and PUK are created for contact use only: the applet's FIPS-mode build accepts a PIN over contactless only inside secure messaging.

Key roles (a bitmask — combine with +, e.g. `AUTHENTICATE+SIGN`, or give a YAML list): `AUTHENTICATE`, `KEY_ESTABLISH`, `SIGN`.

Key attributes (a YAML list): `IMPORTABLE`, `PERMIT_EXTERNAL`, `PERMIT_MUTUAL`, `RSA_CRT`.

PIN/PUK charset (optional, defaults to `numeric`): `numeric`, `alpha`, `alpha_invariant`, `raw`.

> **Note**
>
> No built-in profile creates the Discovery Object container (`oid: 7E`). The object is optional in PIV, and a card without it works everywhere. To add it, put the container in a custom profile (as above) and write the object with `piv perso write-object --object discovery`; it is written and returned in the bare `7E` form SP 800-73-4 requires — the one PIV object that must not be wrapped in a `53` template. Verified on hardware against OpenFIPS201 v2. A malformed (`53`-wrapped) Discovery Object is worse than an absent one: the Windows inbox PIV minidriver rejects the whole card. This tool never writes that form.
>
> The container also cannot be removed afterwards: existing elements cannot be changed or removed, only new ones added, and `load-config` stops at the first element that already exists. A full reset of the PIV applet erases its keys and certificates and runs through the Cryptnox remote PIV service (*Remote service guide*).

`from_yaml` rejects unknown mode/role/mechanism names and validates the whole profile before anything is sent to a card:

- the admin key mechanism must be AES-128/192/256 (this applet's 9B is AES-only);
- PIN and PUK `min` must be at least 6 and `max` must be at least `min`;
- PIN and PUK `retries` must be 0–10;
- every key's mechanism must be one this applet actually supports (see `inspect-defaults`);

- the profile must define at least one container or key.

Generate a starting file with `init-config`, edit it, then apply it with `load-config --file`.

9.1.4 Applying a profile

```
# preview the exact APDUs without touching the card
$ cryptnox-id factory piv preperso load-config --profile cryptnox-default --dry-run

# apply (development/evaluation cards)
$ cryptnox-id factory piv preperso load-config --profile cryptnox-default --default-keys

# or apply a custom profile file
$ cryptnox-id factory piv preperso load-config --file profile.yaml --default-keys
```

Each builder command is sent as a **single short APDU in its own SCP03 session** (JCOP 4.5 accepts one application APDU per applet-directed SCP03 session) — the CLI handles session setup/teardown per command for you.

9.1.5 Next steps

- *PIV personalization* — the full lifecycle this structure feeds into
- *Quick start: a working PIV card* — the condensed, one-command path
- *Factory commands* — the full `factory piv preperso` command reference

9.2 Factory commands

Manufacturing-stage commands. Pre-personalization lays down the PIV applet's *structure* — data containers, PIN/PUK verifiers, key objects — from a profile, over the SCP03 admin channel. Operators normally never need these. As shipped, every card opens the admin channel with the GlobalPlatform test keys (`--default-keys`); replacing them before production is described in *Rotating the PIV admin keys*, after which the commands take the keys from the `PIV_SCP03_ENC` / `PIV_SCP03_MAC` / `PIV_SCP03_DEK` environment variables.

```
factory piv preperso status            lifecycle + structure overview
factory piv preperso inspect-defaults  show the built-in profiles
factory piv preperso init-config       write a built-in profile to editable YAML
factory piv preperso export-config     read-only snapshot of the card's structure
factory piv preperso load-config       apply a profile to the card (supports --dry-run)
factory piv preperso set-mgmt-key      load the PIV management key (9B) value
factory piv preperso finalize          IRREVERSIBLY lock the applet structure
```

`status` never authenticates: the finalize (SECURED) state comes from the applet's own status object, and the admin security domain's SCP version and key versions from its key information template. On this chip an INITIALIZE UPDATE that is not followed by a successful EXTERNAL AUTHENTICATE counts as a failed authentication, so a read-only status must not send one. `piv inventory` lists the structure itself.

`load-config` sends one structural operation per SCP03 session (a platform requirement of this card), so a profile load is a sequence of short commands; it stops at the first rejection and reports exactly what was applied.

The PIV **management key** (key reference `9B`) is the applet's own administration key. Profiles create its key object but never carry a value, because profiles are shareable files; `set-mgmt-key` writes the value over the admin channel from `--default-keys` or `PIV_MGMT_KEY` (`CARD_PIV_MGMT_KEY` is accepted as an alias). With `--default-keys` the published value is written and the variable is not read. It refuses to create a missing `9B` object and refuses to overwrite a value that is already set unless `--replace` is given; `status` shows whether `9B` holds a value.

Built-in profiles: `cryptnox-default` (the applet's own reference structure), `developer` / `npivp-lab` (the same structure, labelled for non-production use), `ssh` (9A gets SIGN added, nothing else changes — see *SSH public-key authentication*), and `ms-logon` (Windows smart-card logon / Remote Desktop: 9A keys are SIGN-capable and an importable RSA-2048 object coexists on 9A — see the *Windows logon & Remote Desktop*).

> **Warning**
>
> `finalize` is **irreversible** — it locks the applet's structure for the card's lifetime. A full reset of the PIV applet runs through the Cryptnox remote PIV service (*Remote service guide*). It is gated by a typed confirmation token when run interactively, or the `--i-understand-this-is-irreversible` flag when run non-interactively — either satisfies the gate. Never run it on a card you are still developing against.

Profiles are documented in *Pre-personalization profiles*.

9.2.1 The card as a whole

```
factory card status   the card's lifecycle and which JCOP modules are loaded
factory card fuse     IRREVERSIBLY fuse the card
```

Both authenticate to the card manager: `--default-keys` for a development card on the GlobalPlatform test keys, otherwise `CARD_MANAGER_ENC` / `CARD_MANAGER_MAC` / `CARD_MANAGER_DEK`. `status` changes nothing; it reports the lifecycle and which of the four modules the card lists. A card can leave these modules out of its listing while they are still loaded, so "not listed" is no proof that a module is gone, and `status` reports the sealed state as unknown unless the configuration module is listed.

`fuse` is the manufacturer's last step. In this order it deletes the JCOP sample modules (`egovaccelerators`, `fips`, `piv`) and then the configuration module, which seals the chip's configuration; moves the card's lifecycle to INITIALIZED and then SECURED, skipping what the card has already reached; and replaces the card-manager key. A DELETE is sent for every module, listed or not, and one that is already gone answers so. The configuration counts as sealed when its DELETE answered that it was deleted or not present. Afterwards it authenticates with the new key and reads the lifecycle again. A fused card still loads and deletes applets for whoever holds the card-manager key; SECURED is the card's normal operating state.

The new key comes from `CARD_MANAGER_NEW_ENC` / `CARD_MANAGER_NEW_MAC` / `CARD_MANAGER_NEW_DEK`, or `--randomize` generates it. `--randomize` requires `--out FILE`: the key file is written with mode 0600, never over an existing file, before anything on the card changes. `--dry-run` shows the plan and needs no current key.

> **Warning**
>
> `fuse` cannot be undone. The configuration stays sealed, the lifecycle cannot go back from SECURED, and the card manager answers only to the new key. The command prints the card's state and a warning, then asks you to type `FUSE-CARD`; `-y` skips the question, and non-interactive use requires `-y`. Keep the new key: without it nobody can administer the card again.

CHAPTER

TEN

LICENSE

This documentation is licensed under CC BY-NC-ND 4.0.

- **Attribution** for appropriate credit to Cryptnox SA
- **NonCommercial** for non-commercial use only
- **NoDerivatives** for unmodified redistribution only

For inquiries, contact: contact@cryptnox.com

```
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0
International Public License

By exercising the Licensed Rights (defined below), You accept and agree
to be bound by the terms and conditions of this Creative Commons
Attribution-NonCommercial-NoDerivatives 4.0 International Public
License ("Public License"). To the extent this Public License may be
interpreted as a contract, You are granted the Licensed Rights in
consideration of Your acceptance of these terms and conditions, and the
Licensor grants You such rights in consideration of benefits the
Licensor receives from making the Licensed Material available under
these terms and conditions.


Section 1 -- Definitions.

  a. Adapted Material means material subject to Copyright and Similar
     Rights that is derived from or based upon the Licensed Material
     and in which the Licensed Material is translated, altered,
     arranged, transformed, or otherwise modified in a manner requiring
     permission under the Copyright and Similar Rights held by the
     Licensor. For purposes of this Public License, where the Licensed
     Material is a musical work, performance, or sound recording,
     Adapted Material is always produced where the Licensed Material is
     synched in timed relation with a moving image.

  b. Copyright and Similar Rights means copyright and/or similar rights
     closely related to copyright including, without limitation,
     performance, broadcast, sound recording, and Sui Generis Database
     Rights, without regard to how the rights are labeled or
     categorized. For purposes of this Public License, the rights
     specified in Section 2(b)(1)-(2) are not Copyright and Similar
```

(continues on next page)

(continued from previous page)

```
     Rights.

  c. Effective Technological Measures means those measures that, in the
     absence of proper authority, may not be circumvented under laws
     fulfilling obligations under Article 11 of the WIPO Copyright
     Treaty adopted on December 20, 1996, and/or similar international
     agreements.

  d. Exceptions and Limitations means fair use, fair dealing, and/or
     any other exception or limitation to Copyright and Similar Rights
     that applies to Your use of the Licensed Material.

  e. Licensed Material means the artistic or literary work, database,
     or other material to which the Licensor applied this Public
     License.

  f. Licensed Rights means the rights granted to You subject to the
     terms and conditions of this Public License, which are limited to
     all Copyright and Similar Rights that apply to Your use of the
     Licensed Material and that the Licensor has authority to license.

  g. Licensor means the individual(s) or entity(ies) granting rights
     under this Public License.

  h. NonCommercial means not primarily intended for or directed towards
     commercial advantage or monetary compensation. For purposes of
     this Public License, the exchange of the Licensed Material for
     other material subject to Copyright and Similar Rights by digital
     file-sharing or similar means is NonCommercial provided there is
     no payment of monetary compensation in connection with the
     exchange.

  i. Share means to provide material to the public by any means or
     process that requires permission under the Licensed Rights, such
     as reproduction, public display, public performance, distribution,
     dissemination, communication, or importation, and to make material
     available to the public including in ways that members of the
     public may access the material from a place and at a time
     individually chosen by them.

  j. Sui Generis Database Rights means rights other than copyright
     resulting from Directive 96/9/EC of the European Parliament and of
     the Council of 11 March 1996 on the legal protection of databases,
     as amended and/or succeeded, as well as other essentially
     equivalent rights anywhere in the world.

  k. You means the individual or entity exercising the Licensed Rights
     under this Public License. Your has a corresponding meaning.


Section 2 -- Scope.
```

(continues on next page)

(continued from previous page)

```
a. License grant.

     1. Subject to the terms and conditions of this Public License,
        the Licensor hereby grants You a worldwide, royalty-free,
        non-sublicensable, non-exclusive, irrevocable license to
        exercise the Licensed Rights in the Licensed Material to:

          a. reproduce and Share the Licensed Material, in whole or
             in part, for NonCommercial purposes only; and

          b. produce and reproduce, but not Share, Adapted Material
             for NonCommercial purposes only.

     2. Exceptions and Limitations. For the avoidance of doubt, where
        Exceptions and Limitations apply to Your use, this Public
        License does not apply, and You do not need to comply with
        its terms and conditions.

     3. Term. The term of this Public License is specified in Section
        6(a).

     4. Media and formats; technical modifications allowed. The
        Licensor authorizes You to exercise the Licensed Rights in
        all media and formats whether now known or hereafter created,
        and to make technical modifications necessary to do so. The
        Licensor waives and/or agrees not to assert any right or
        authority to forbid You from making technical modifications
        necessary to exercise the Licensed Rights, including
        technical modifications necessary to circumvent Effective
        Technological Measures. For purposes of this Public License,
        simply making modifications authorized by this Section 2(a)
        (4) never produces Adapted Material.

     5. Downstream recipients.

          a. Offer from the Licensor -- Licensed Material. Every
             recipient of the Licensed Material automatically
             receives an offer from the Licensor to exercise the
             Licensed Rights under the terms and conditions of this
             Public License.

          b. No downstream restrictions. You may not offer or impose
             any additional or different terms or conditions on, or
             apply any Effective Technological Measures to, the
             Licensed Material if doing so restricts exercise of the
             Licensed Rights by any recipient of the Licensed
             Material.

     6. No endorsement. Nothing in this Public License constitutes or
        may be construed as permission to assert or imply that You
        are, or that Your use of the Licensed Material is, connected
        with, or sponsored, endorsed, or granted official status by,
```

(continues on next page)

(continued from previous page)

```
          the Licensor or others designated to receive attribution as
          provided in Section 3(a)(1)(A)(i).

  b. Other rights.

       1. Moral rights, such as the right of integrity, are not
          licensed under this Public License, nor are publicity,
          privacy, and/or other similar personality rights; however, to
          the extent possible, the Licensor waives and/or agrees not to
          assert any such rights held by the Licensor to the limited
          extent necessary to allow You to exercise the Licensed
          Rights, but not otherwise.

       2. Patent and trademark rights are not licensed under this
          Public License.

       3. To the extent possible, the Licensor waives any right to
          collect royalties from You for the exercise of the Licensed
          Rights, whether directly or through a collecting society
          under any voluntary or waivable statutory or compulsory
          licensing scheme. In all other cases the Licensor expressly
          reserves any right to collect such royalties, including when
          the Licensed Material is used other than for NonCommercial
          purposes.


Section 3 -- License Conditions.

Your exercise of the Licensed Rights is expressly made subject to the
following conditions.

  a. Attribution.

       1. If You Share the Licensed Material, You must:

            a. retain the following if it is supplied by the Licensor
               with the Licensed Material:

                 i. identification of the creator(s) of the Licensed
                    Material and any others designated to receive
                    attribution, in any reasonable manner requested by
                    the Licensor (including by pseudonym if
                    designated);

                ii. a copyright notice;

               iii. a notice that refers to this Public License;

                iv. a notice that refers to the disclaimer of
                    warranties;

                 v. a URI or hyperlink to the Licensed Material to the
```

(continues on next page)

(continued from previous page)

```
                    extent reasonably practicable;

            b. indicate if You modified the Licensed Material and
               retain an indication of any previous modifications; and

            c. indicate the Licensed Material is licensed under this
               Public License, and include the text of, or the URI or
               hyperlink to, this Public License.

          For the avoidance of doubt, You do not have permission under
          this Public License to Share Adapted Material.

       2. You may satisfy the conditions in Section 3(a)(1) in any
          reasonable manner based on the medium, means, and context in
          which You Share the Licensed Material. For example, it may be
          reasonable to satisfy the conditions by providing a URI or
          hyperlink to a resource that includes the required
          information.

       3. If requested by the Licensor, You must remove any of the
          information required by Section 3(a)(1)(A) to the extent
          reasonably practicable.


Section 4 -- Sui Generis Database Rights.

Where the Licensed Rights include Sui Generis Database Rights that
apply to Your use of the Licensed Material:

  a. for the avoidance of doubt, Section 2(a)(1) grants You the right
     to extract, reuse, reproduce, and Share all or a substantial
     portion of the contents of the database for NonCommercial purposes
     only and provided You do not Share Adapted Material;

  b. if You include all or a substantial portion of the database
     contents in a database in which You have Sui Generis Database
     Rights, then the database in which You have Sui Generis Database
     Rights (but not its individual contents) is Adapted Material; and

  c. You must comply with the conditions in Section 3(a) if You Share
     all or a substantial portion of the contents of the database.

For the avoidance of doubt, this Section 4 supplements and does not
replace Your obligations under this Public License where the Licensed
Rights include other Copyright and Similar Rights.


Section 5 -- Disclaimer of Warranties and Limitation of Liability.

  a. UNLESS OTHERWISE SEPARATELY UNDERTAKEN BY THE LICENSOR, TO THE
     EXTENT POSSIBLE, THE LICENSOR OFFERS THE LICENSED MATERIAL AS-IS
     AND AS-AVAILABLE, AND MAKES NO REPRESENTATIONS OR WARRANTIES OF
```

(continues on next page)

(continued from previous page)

```
     ANY KIND CONCERNING THE LICENSED MATERIAL, WHETHER EXPRESS,
     IMPLIED, STATUTORY, OR OTHER. THIS INCLUDES, WITHOUT LIMITATION,
     WARRANTIES OF TITLE, MERCHANTABILITY, FITNESS FOR A PARTICULAR
     PURPOSE, NON-INFRINGEMENT, ABSENCE OF LATENT OR OTHER DEFECTS,
     ACCURACY, OR THE PRESENCE OR ABSENCE OF ERRORS, WHETHER OR NOT
     KNOWN OR DISCOVERABLE. WHERE DISCLAIMERS OF WARRANTIES ARE NOT
     ALLOWED IN FULL OR IN PART, THIS DISCLAIMER MAY NOT APPLY TO YOU.

  b. TO THE EXTENT POSSIBLE, IN NO EVENT WILL THE LICENSOR BE LIABLE
     TO YOU ON ANY LEGAL THEORY (INCLUDING, WITHOUT LIMITATION,
     NEGLIGENCE) OR OTHERWISE FOR ANY DIRECT, SPECIAL, INDIRECT,
     INCIDENTAL, CONSEQUENTIAL, PUNITIVE, EXEMPLARY, OR OTHER LOSSES,
     COSTS, EXPENSES, OR DAMAGES ARISING OUT OF THIS PUBLIC LICENSE OR
     USE OF THE LICENSED MATERIAL, EVEN IF THE LICENSOR HAS BEEN
     ADVISED OF THE POSSIBILITY OF SUCH LOSSES, COSTS, EXPENSES, OR
     DAMAGES. WHERE A LIMITATION OF LIABILITY IS NOT ALLOWED IN FULL OR
     IN PART, THIS LIMITATION MAY NOT APPLY TO YOU.

  c. The disclaimer of warranties and limitation of liability provided
     above shall be interpreted in a manner that, to the extent
     possible, most closely approximates an absolute disclaimer and
     waiver of all liability.


Section 6 -- Term and Termination.

  a. This Public License applies for the term of the Copyright and
     Similar Rights licensed here. However, if You fail to comply with
     this Public License, then Your rights under this Public License
     terminate automatically.

  b. Where Your right to use the Licensed Material has terminated under
     Section 6(a), it reinstates:

       1. automatically as of the date the violation is cured, provided
          it is cured within 30 days of Your discovery of the
          violation; or

       2. upon express reinstatement by the Licensor.

     For the avoidance of doubt, this Section 6(b) does not affect any
     right the Licensor may have to seek remedies for Your violations
     of this Public License.

  c. For the avoidance of doubt, the Licensor may also offer the
     Licensed Material under separate terms or conditions or stop
     distributing the Licensed Material at any time; however, doing so
     will not terminate this Public License.

  d. Sections 1, 5, 6, 7, and 8 survive termination of this Public
     License.
```

(continues on next page)

(continued from previous page)

```
Section 7 -- Other Terms and Conditions.

  a. The Licensor shall not be bound by any additional or different
     terms or conditions communicated by You unless expressly agreed.

  b. Any arrangements, understandings, or agreements regarding the
     Licensed Material not stated herein are separate from and
     independent of the terms and conditions of this Public License.


Section 8 -- Interpretation.

  a. For the avoidance of doubt, this Public License does not, and
     shall not be interpreted to, reduce, limit, restrict, or impose
     conditions on any use of the Licensed Material that could lawfully
     be made without permission under this Public License.

  b. To the extent possible, if any provision of this Public License is
     deemed unenforceable, it shall be automatically reformed to the
     minimum extent necessary to make it enforceable. If the provision
     cannot be reformed, it shall be severed from this Public License
     without affecting the enforceability of the remaining terms and
     conditions.

  c. No term or condition of this Public License will be waived and no
     failure to comply consented to unless expressly agreed to by the
     Licensor.

  d. Nothing in this Public License constitutes or may be interpreted
     as a limitation upon, or waiver of, any privileges and immunities
     that apply to the Licensor or You, including from the legal
     processes of any jurisdiction or authority.
```

CHAPTER

ELEVEN

QUICK START: A WORKING PIV CARD

This guide takes a blank (pre-personalized) Cryptnox card to a PIV credential that standard tooling — `yubico-piv-tool`, OpenSC/PKCS#11 — can read, verify and sign with. About ten minutes, one command if you let it.

11.1 Prerequisites

- A **contact** smart-card reader. The PIN, the PUK and the PIN-protected keys work over the contact interface only, and this guide presents the PIN partway through. Administration works over both interfaces; apart from it, a contactless reader gets just the data that needs no PIN, such as the CHUID.
- `cryptnox-id` installed, and `yubico-piv-tool` if you want to run the verification step (any recent version).
- The card's SCP03 administration keys. As shipped, every card uses the GlobalPlatform test keys — pass `--default-keys`. Before a card goes into production, replace them (*Rotating the PIV admin keys*); from then on the keys come from the `PIV_SCP03_ENC` / `PIV_SCP03_MAC` / `PIV_SCP03_DEK` environment variables (hex). Keys never go on the command line — see the warning in *Getting started*.

Check what you have:

```
$ cryptnox-id readers
$ cryptnox-id piv status
```

11.2 The one-command path

```
$ cryptnox-id piv quickstart --default-keys --dry-run   # show the plan first
$ cryptnox-id piv quickstart --default-keys
```

`piv quickstart` detects the card state, then runs whatever is still needed of: PIN -> PUK -> key in slot 9C (ECC P-256, generated **on-card** — the private key never exists outside the card) -> self-signed certificate -> CHUID/CCC -> smoke-test. Steps already done are skipped, so re-runs converge. New PIN/PUK values come from masked prompts or `CRYPTNOX_PIV_NEW_PIN` / `CRYPTNOX_PIV_NEW_PUK`.

On a factory-blank card that has never had its PIV structure laid down, add `--include-preperso` (a manufacturing-style step, intended for development cards), and for a CA-issued certificate instead of a self-signed one use `--cert-mode csr --csr-out 9c.csr.pem`.

11.3 The steps, by hand

The same sequence as individual commands — useful to understand what quickstart does, or to vary it.

Step 0 — lay down the PIV structure (once per card). If `piv status` reports a pre-personalized applet with no containers/verifiers:

```
$ cryptnox-id factory piv preperso load-config --profile cryptnox-default --dry-run
$ cryptnox-id factory piv preperso load-config --profile cryptnox-default --default-keys
```

Step 1 — set the PIN and PUK (values prompted, never echoed):

```
$ cryptnox-id piv perso set-puk --default-keys
$ cryptnox-id piv perso set-pin --default-keys
$ cryptnox-id piv pin status
```

Step 2 — put a key in slot 9C. Slot 9C (Digital Signature) is the SIGN-role slot on this card — certificate signing needs it (9A is AUTHENTICATE-role and returns `6985` for sign operations).

```
$ cryptnox-id piv perso generate-key --slot 9C --algorithm ECCP256 \
    --out 9c.pub.pem --default-keys
```

Note

`quickstart` defaults to **ECC** (P-256), except with `--profile ms-logon` on slot 9A, the one built-in profile/slot combination with an RSA key object, where it defaults to RSA-2048 because the Windows inbox PIV minidriver does not enumerate EC keys (see *Windows logon & Remote Desktop*). Elsewhere `--algorithm RSA2048` is accepted only when the card already has an RSA-2048 key object for the slot.

A slot can be generated on-card with any mechanism it has a **key object** for, and the pre-personalization profile decides which those are. Every key object in `cryptnox-default` is ECC, so RSA must be imported on such a card. A profile that provides an RSA object (`ms-logon` does, on 9A) can generate RSA on-card instead, with the private key never leaving the chip:

```
$ cryptnox-id piv perso generate-key --slot 9A --algorithm RSA2048 \
    --out 9a.pub.pem
```

Asking for a mechanism the slot has no object for returns `6A80`.

To place an **externally generated** RSA key instead:

```
$ openssl genrsa -out rsa2048.key.pem 2048
$ cryptnox-id piv perso import-key --slot 9C --key rsa2048.key.pem \
    --create-key-object --default-keys
```

`import-key` also accepts ECC keys (escrow/migration scenarios). See *PIV personalization*.

Step 3 — a certificate for 9C. Fastest (development/testing):

```
$ cryptnox-id piv perso self-sign-cert --slot 9C --subject "CN=Test User" \
    --public-key 9c.pub.pem --out 9c.crt.pem
$ cryptnox-id piv perso import-cert --slot 9C --cert 9c.crt.pem
```

Or CA-issued: `generate-csr` -> have your CA sign it -> `import-cert --csr 9c.csr.pem`, which refuses a certificate the CA issued for some other key. Large certificates are written with ISO command chaining automatically.

Step 4 — standard data objects. Most PIV clients (including `yubico-piv-tool -a status` and the Windows minidriver) treat a card without a CHUID as unpersonalized:

```
$ cryptnox-id piv perso write-standard-objects --default-keys
```

Step 5 — validate from our side:

```
$ cryptnox-id piv perso smoke-test     # verify PIN + one on-card signature
$ cryptnox-id piv validate             # consistency check (NOT a FIPS/NIST validation)
```

11.4 Verify with yubico-piv-tool

> **Note**
>
> `yubico-piv-tool` auto-connects only to readers whose name contains "Yubikey" — always pass your reader with `-r` (any unique substring of the reader name works). `ACR39` below matches the verified ACS contact reader; substitute your own reader's name as shown by `cryptnox-id readers` — a Cryptnox reader shows either its Cryptnox name (`CryptnoxCR` contact, `Cryptnox NFC` contactless) or the ACS model name of the unit.

```
$ yubico-piv-tool -r "ACR39" -a status
$ yubico-piv-tool -r "ACR39" -a read-certificate -s 9c -o 9c-readback.pem
$ yubico-piv-tool -r "ACR39" -a verify-pin
```

For PKCS#11 (SSH, TLS client auth, code signing through OpenSC):

```
$ pkcs11-tool --module opensc-pkcs11 --list-objects
$ pkcs15-tool --list-certificates
```

11.5 Interoperability matrix

The card is a standards-compliant SP 800-73 PIV target for *reading, PIN operations and signing*. *Provisioning* is done with `cryptnox-id` over an SCP03 secure channel, by design — Yubico-proprietary management does not apply.

> **Note**
>
> `yubico-piv-tool` checks the Yubico firmware-version query **at connect time** and refuses any card that does not answer it ("Failed to connect to yubikey: Not supported."). The rows below were validated against a card whose applet includes that compatibility responder; a card running the plain PIV applet without it is refused outright. It is the only tool with that requirement: OpenSC and the Windows smart-card stack use standard PIV discovery and work against the PIV function regardless.

yubico-piv-tool action	On this card	Why
`-a status`	works	Standard SELECT + GET DATA. Containers that hold no parseable certificate print cosmetic `Parse error` lines — populated slots render normally.
`-a read-certificate -s 9c`	works	Standard GET DATA on the certificate object
`-a version`	works (with the compatibility responder — see note)	Answers the Yubico version query with a YubiKey-compatible firmware version; the tool requires this to connect at all
`-a verify-pin`	works	Standard VERIFY
`-a change-pin` / `-a change-puk` / `-a unblock-pin`	works	Standard CHANGE REFERENCE DATA / RESET RETRY COUNTER
OpenSC: `pkcs15-tool --list-certificates`, `pkcs15-crypt --sign`, PKCS#11 signing	works	Pure SP 800-73: OpenSC identifies the card as a "Personal Identity Verification Card" and signs with the slot key after PIN verification — no vendor extensions involved
`-a attest`	not applicable	Yubico vendor attestation; no attestation key/certificate on this applet (`Failed to attest data.`)
`-a set-mgm-key`	not applicable	Yubico management-key model; this card administers over a GlobalPlatform SCP03 channel instead
`-a authenticate` + writes (`-a generate`, `-a import-key`, `-a import-certificate`, `-a set-chuid`)	use `cryptnox-id` instead	Writes require Yubico-style management-key authentication; on this card every write goes over SCP03 via `cryptnox-id piv perso ...`

11.6 If something fails

Symptom	Cause / fix
`6985` when signing or self-signing	Wrong slot role: sign operations need a key with the SIGN role (9C on `cryptnox-default`); the error names the role, and `piv perso create-key-object` adds a signing key object for another algorithm
`set-pin` fails `6A88`/`6A82`	The PIV structure is missing — run step 0
yubico-piv-tool sees no card	Pass the reader explicitly: `-r <substring>`
A PIN step fails with `6982` over NFC	The PIN is contact-only — move the card to a contact reader
SCP03 mutual authentication fails	Wrong admin keys — `--default-keys` is for development cards only

More: *Troubleshooting*.

11.7 Next steps

- *PIV personalization* — the full lifecycle, key import, finalize
- *Pre-personalization profiles* — customizing the pre-personalization profile
- *PIV commands* — every `piv` command

CHAPTER

TWELVE

WINDOWS LOGON & REMOTE DESKTOP

Remote Desktop with a smart card is **Windows smart-card logon**: `mstsc` redirects the card into the session and the remote host performs Kerberos PKINIT against a certificate on the card. This guide personalizes the card for that — and for interactive Windows logon generally.

12.1 How it has to look

Windows reads the logon certificate from slot **9A** (PIV Authentication) via the inbox PIV minidriver. The applet only performs client-authentication signing for keys created with a SIGN-capable role, so the card must be pre-personalized with the **ms-logon profile** (its 9A keys are `AUTHENTICATE+SIGN`, and it adds an importable RSA-2048 object on 9A).

Logon needs a **contact** reader, or the contact slot of a combo reader. Over contactless this applet accepts a PIN only inside PIV secure messaging (the SP 800-73-4 virtual contact interface); the built-in profiles create the PIN for contact use only, and the inbox minidriver has no documented support for secure messaging.

The certificate itself is dictated by Active Directory, not by the card:

- issued by the **enterprise CA** (present in the domain's NTAuth store) — a self-signed certificate can never log on;
- from a smart-card-logon template: *Client Authentication* and *Smart Card Logon* EKUs and the user's **UPN in the Subject Alternative Name**;
- since the KB5014754 strong-mapping enforcement, carrying the **SID security extension** — Active Directory Certificate Services adds it automatically when issuing against a user from a current template;
- the key in 9A must be **RSA** — see below.

Important

Use an RSA key on Windows. The inbox PIV minidriver does not enumerate EC keys.

With an EC key in 9A, Windows does not merely reject the certificate, it never builds a key container at all: `certutil -scinfo` reports `NTE_BAD_KEYSET`, nothing appears in the Windows certificate store, and no application can offer the credential. The card looks empty rather than broken, which makes this easy to misdiagnose.

This is a property of the Windows driver, not of this card. The same test reproduces on a YubiKey 5, and P-384 fails the same way as P-256.

Because of this, `piv quickstart` defaults to RSA-2048 on the `ms-logon` profile with slot 9A, the Windows shape, and to ECC everywhere else. The examples below pass `--algorithm RSA2048` all the same, so a copied command stays right if the shape changes. The RSA key is still generated **on-card** and never leaves it, so choosing RSA costs nothing in key protection.

If a deployment genuinely requires ECC on Windows, that needs a third-party minidriver (Yubico's supports P-256/P-384), which is outside the no-extra-software path this guide describes.

12.2 Step 0 — pre-personalize with the ms-logon profile (once per card)

```
$ cryptnox-id factory piv preperso load-config --profile ms-logon --default-keys
$ cryptnox-id piv quickstart --profile ms-logon --include-preperso \
    --slot 9A --algorithm RSA2048 --cert-mode csr --csr-out 9a.csr.pem --default-keys
```

The second form does everything in one shot — PIN, PUK, an on-card **RSA-2048** key in 9A (the private key never leaves the card), a CSR for your CA, and the CHUID/CCC objects Windows expects. It reports the key type it chose as it runs.

12.3 Step 1 — get the certificate issued

Two paths; pick the one your PKI supports. Path A2 is Path A run by an administrator for another user.

Path A — CSR to the CA (key stays on-card). Submit `9a.csr.pem` against a smart-card-logon template and import the issued certificate:

```
$ certreq -submit -attrib "CertificateTemplate:SmartcardLogon" 9a.csr.pem 9a-issued.cer
$ cryptnox-id piv perso import-cert --slot 9A --cert 9a-issued.cer --csr 9a.csr.pem
```

`--csr` makes the import refuse a certificate whose public key is not the one in the CSR, which is what happens when an enrollment tool generated a key of its own instead of signing the card's request. A certificate for another key would be written without complaint and fail only at logon.

Path A2: enroll on behalf of another user (enrollment agent). An administrator provisioning a card for another Active Directory user cannot use the MMC *Enroll on behalf of* wizard: it generates a new key in software, so the issued certificate does not match the key in 9A. Instead, wrap the card's CSR in a CMC request that names the user and is signed with the administrator's enrollment agent certificate, and submit that. The key stays on the card.

The CA needs, once:

- an enrollment agent certificate for the administrator, from the *Enrollment Agent* template (extended key usage *Certificate Request Agent*, `1.3.6.1.4.1.311.20.2.1`), with its private key in the administrator's certificate store;
- a smart-card-logon template, schema version 2 or later, with one authorized signature and the application policy *Certificate Request Agent* under *Issuance Requirements*, the subject built from Active Directory with the UPN in the alternative name, Enroll permission for the enrollment agent, private key archival off, and a minimum key size no larger than the card's key (2048);
- if the CA restricts enrollment agents (CA properties, *Enrollment Agents*), an entry that allows this agent for this template and these users.

On the administrator's Windows machine, find the agent certificate's thumbprint:

```
Get-ChildItem Cert:\CurrentUser\My |
    Where-Object { $_.EnhancedKeyUsageList.ObjectId -contains '1.3.6.1.4.1.311.20.2.1' }␣
↪|
    Format-List Subject, Thumbprint, NotAfter
```

Wrap the CSR from Step 0 with `eobo-wrap.ps1` (Windows PowerShell and the built-in CertEnroll COM objects, nothing to install) and check the result:

```
.\eobo-wrap.ps1 -CsrPath .\9a.csr.pem -RequesterName "CONTOSO\jdoe" -AgentThumbprint
↪<thumbprint>
certutil -dump .\eobo-cmc.req
```

`-MachineStore` takes the agent certificate from the computer's store instead of the user's. `certutil -dump` should show the requester name, the card's request unchanged inside, and the agent certificate as the signer. Submit the request and import the issued certificate:

```
$ certreq -submit -config "<CA host>\<CA name>" -attrib "CertificateTemplate:<template>"␣
↪eobo-cmc.req jdoe.cer
$ cryptnox-id piv perso import-cert --slot 9A --cert jdoe.cer --csr 9a.csr.pem
```

As in Path A, `--csr` refuses a certificate issued for another key.

> **Note**
>
> The wrap is tested on Windows: `certutil -dump` shows the requester name, the unchanged inner request and the agent's signature. Submission to a CA and logon with the issued certificate are not tested.

The script:

```
param(
  [Parameter(Mandatory=$true)][string]$CsrPath,
  [Parameter(Mandatory=$true)][string]$RequesterName,
  [Parameter(Mandatory=$true)][string]$AgentThumbprint,
  [string]$OutPath = ".\eobo-cmc.req",
  [switch]$MachineStore
)
$ErrorActionPreference = 'Stop'
$store = if ($MachineStore) { "LocalMachine" } else { "CurrentUser" }
$agent = Get-Item ("Cert:\$store\My\" + ($AgentThumbprint -replace '\s',''))
if (-not $agent.HasPrivateKey) { throw "Enrollment Agent certificate has no private key␣
↪in $store\My" }
$inner = New-Object -ComObject X509Enrollment.CX509CertificateRequestPkcs10
$inner.InitializeDecode((Get-Content $CsrPath -Raw), 6)
$cmc = New-Object -ComObject X509Enrollment.CX509CertificateRequestCmc
$cmc.InitializeFromInnerRequest($inner)
$cmc.RequesterName = $RequesterName
$signer = New-Object -ComObject X509Enrollment.CSignerCertificate
$signer.Initialize([bool]$MachineStore, 0, 1, [Convert]::ToBase64String($agent.RawData))
$cmc.SignerCertificate = $signer
$cmc.Encode()
$b64 = $cmc.RawData(3)
[IO.File]::WriteAllText($ExecutionContext.SessionState.Path.
↪GetUnresolvedProviderPathFromPSPath($OutPath), $b64)
Write-Host "CMC request written to $OutPath (requester $RequesterName, signed by $(
↪$agent.Subject))"
```

Path B — PKI-issued PKCS#12 (centralized key generation). If your PKI delivers a `.pfx`/`.p12`, one command imports both the key and the certificate:

```
$ cryptnox-id piv perso import-p12 --slot 9A --p12 user.pfx
```

(The container password comes from a masked prompt or `CRYPTNOX_PIV_P12_PASSWORD`; chain certificates inside the container are ignored — PIV stores the entity certificate.)

12.4 Step 2 — verify Windows sees the card

```
$ certutil -scinfo
```

You should see the card bind to the PIV minidriver ("Identity Device (NIST SP 800-73 [PIV])") and the 9A certificate listed.

Windows logon, like OpenSC, uses **standard PIV discovery and standard PIV commands only** — an `ms-logon` card lists its certificate and produces a PIN-verified signature with the 9A key under OpenSC (tested with the ECC key on 9A; on-card signing with the RSA-2048 key is separately verified via the CLI's smoke test). The Yubico-specific connect requirement of `yubico-piv-tool` (see the *Quick start: a working PIV card* interop note) plays no role in logon.

> **Note**
>
> **If another vendor's smart-card middleware is installed** (SafeNet/Thales, etc.), its ATR mask may claim this card before Windows runs PIV discovery, handing it to the wrong driver. The fix is an administrator-level smart card registry association (Calais database) mapping this card's exact ATR to the PIV class minidriver — reversible, per machine. On machines without competing middleware, no configuration is needed.

12.5 Step 3 — log on

- **Local/interactive**: select the smart-card credential tile and enter the PIV PIN.
- **Remote Desktop**: in `mstsc` the card is redirected by default (Local Resources -> Smart cards). Connect to the domain host and authenticate with the PIN at the remote credential prompt.

12.6 Scope and limits

- Smart-card logon requires a **domain** (Active Directory or hybrid-joined Entra ID). Standalone Windows machines do not support smart-card logon natively.
- Enrollment runs through `cryptnox-id` (CSR or PKCS#12). Windows-side enrollment tools that write directly to the card (certreq onto the card, MMC enrollment, *Enroll on behalf of*) use the Yubico-style management model and do not apply; this card administers over SCP03. To enroll for another user, use Path A2.
- The end-to-end logon path (PKINIT against a domain controller) can only be proven in a domain lab; the card-side personalization and minidriver recognition are verifiable standalone with `certutil -scinfo`.

12.7 Next steps

- *Quick start: a working PIV card* — the general PIV quick start and tooling interop
- *Pre-personalization profiles* — what the ms-logon profile defines
- *PIV commands* — `import-key`, `import-p12`, `quickstart`

CHAPTER

THIRTEEN

SSH PUBLIC-KEY AUTHENTICATION

SSH via PIV means the private key never leaves the card: `ssh-agent` loads it through PKCS#11 (OpenSC), signs the challenge on-card after a PIN prompt, and the server authorizes it like any other public key in `authorized_keys`. No card-specific server-side support needed.

Note

Everything below that references `cryptnox-default` applies to a card provisioned with that specific built-in profile — one option among several (see *Pre-personalization profiles*), not necessarily what any given card actually has. Check which profile your card was provisioned with (`factory piv preperso export-config`, or ask whoever personalized it) before assuming this section applies to you.

Warning

9C cannot be used for SSH on the ``cryptnox-default`` profile. Steps 3/4 below fail with `Permission denied (publickey)` / `agent refused operation` even with the correct PIN, with or without `ssh-agent` in the loop — see *Why it fails on cryptnox-default*. Use **9A with the SIGN role added** instead — the built-in `ssh` profile does exactly this — see *Fixing it: give 9A the SIGN role* below.

13.1 How it has to look

The applet only performs challenge-response signing (what SSH auth needs) for keys created with a **SIGN-capable role**. On `cryptnox-default` that's slot 9C — but 9C's PIN policy makes it unusable for SSH regardless (see below). 9A (PIV Authentication) is `AUTHENTICATE`-role only on `cryptnox-default` and returns `6985` on a sign attempt, so it needs a profile change before it can be used — the built-in **ssh** profile changes only 9A's role (no extra objects, unlike **ms-logon**, which also adds `SIGN+AUTHENTICATE` to 9A but includes an importable RSA-2048 object for AD-issued credentials you don't need here) — see *Pre-personalization profiles* and *Fixing it: give 9A the SIGN role*.

This guide's steps below use whichever slot has your key + certificate — *Quick start: a working PIV card* puts them on 9C by default, which per the warning above will not work for SSH. Provision 9A instead (next section) before following Steps 0–4.

13.2 Why it fails on cryptnox-default

- `pkcs11-tool --login --sign` against the 9C key succeeds, but prompts for the PIN **twice** — once for normal login, then again for "context specific PIN". That second prompt is PKCS#11's `CKA_ALWAYS_AUTHENTICATE`: a fresh re-login is required immediately before *every single* signing operation, not just once per session.

- `ssh-add -s` then `ssh -I` (agent-mediated) fails: the agent only ever prompts once, at load time, and has no way to supply that second re-authentication later when a connection actually needs a signature — hence `agent refused operation`.
- Bypassing the agent entirely — `ssh -I "$PKCS11_MODULE"` run directly, foreground, no agent involved — fails the same way. OpenSSH's own built-in PKCS#11 client only performs the normal login, not the context-specific re-login `CKA_ALWAYS_AUTHENTICATE` demands before signing. Only tools that explicitly implement that second step (like `pkcs11-tool`) can use this key at all.

Why it has to be 9A specifically: `pkcs15-tool --dump` shows why 9C behaves this way — its private key object reports `Usage: [0x204], sign, nonRepudiation`. The `nonRepudiation` bit is what triggers OpenSC's always-reauth handling; it matches PIV convention (slot 9C, "Digital Signature", is spec'd for non-repudiation, so the standard expects fresh PIN verification before each signature). This is assigned by OpenSC's PIV driver for key reference 0x9C specifically, independent of the card's own profile or certificate — 9A's private key object reports plain `Usage: [0x04], sign`, with no `nonRepudiation` bit, so it is not subject to the always-reauth requirement. 9A is also the slot PIV itself designates for cardholder-to-system authentication, and its access mode on `cryptnox-default` requires a PIN — **over the contact interface only**; 9A's contactless access mode is `NEVER`, so none of this works over a contactless-only reader (see Prerequisites below).

13.3 Fixing it: give 9A the SIGN role

You don't need the full `ms-logon` profile for this — the built-in `ssh` profile changes only 9A's `role` (`AUTHENTICATE -> AUTHENTICATE, SIGN`); everything else is identical to `cryptnox-default`. See *Pre-personalization profiles* for the full built-in profile list.

The profile applies at **pre-personalization**: it sets the key's role at the structural level, and an existing key object keeps its role. If your card already has a different profile applied (e.g. it already went through *Quick start: a working PIV card* on 9C), this profile cannot be loaded over it. Give 9A a second key object for another algorithm with the SIGN role instead, then give it a key and a certificate, with the card in a contact reader:

```
$ cryptnox-id piv perso create-key-object --slot 9A --algorithm ECCP384 \
    --role SIGN+AUTHENTICATE --default-keys
$ cryptnox-id piv perso generate-key --slot 9A --algorithm ECCP384 \
    --out 9a.pub.pem --default-keys
$ cryptnox-id piv perso self-sign-cert --slot 9A --algorithm ECCP384 \
    --subject "CN=Test User" --public-key 9a.pub.pem --out 9a.crt.pem
$ cryptnox-id piv perso import-cert --slot 9A --cert 9a.crt.pem --default-keys
$ cryptnox-id piv perso smoke-test --slot 9A --algorithm ECCP384
```

The object is permanent, and the first 9A key stays on the card, unused (see *PIV commands*). `generate-key --create-key-object` is not the way here: it copies the default profile's role, AUTHENTICATE only on 9A. The block below is for a blank or pre-personalized card; skip it on this path.

> **Warning**
>
> Run the whole block below with the card in a **contact** reader. The PIN, the PUK and the 9A key are contact-only on every built-in profile. Over a contactless (NFC/PICC) reader the first four commands still succeed, since they run inside the admin channel, and `self-sign-cert` is the first one to present the PIN. It fails there with `PIN verification refused (SW=6982)` while `piv pin status` reports the PIN as not usable over this interface. Nothing is wrong with the card and the PIN you set is kept: move the card to a contact reader and continue from `self-sign-cert`.

```
$ cryptnox-id factory piv preperso load-config --profile ssh --default-keys
$ cryptnox-id piv perso set-puk --default-keys
$ cryptnox-id piv perso set-pin --default-keys
$ cryptnox-id piv perso generate-key --slot 9A --algorithm ECCP256 \
    --out 9a.pub.pem --default-keys
$ cryptnox-id piv perso self-sign-cert --slot 9A --subject "CN=Test User" \
    --public-key 9a.pub.pem --out 9a.crt.pem
$ cryptnox-id piv perso import-cert --slot 9A --cert 9a.crt.pem --default-keys
$ cryptnox-id piv perso write-standard-objects --default-keys
```

Warning

Every step above must explicitly say `--slot 9A`. *Quick start: a working PIV card* defaults everywhere to 9C, and copy-pasting those defaults here silently provisions the wrong slot.

Note

`piv perso smoke-test` defaults to `--slot 9C`. On this profile 9C was never populated, so the plain command fails with "the ECCP256 key object in slot 9C holds no key yet". Always run `piv perso smoke-test --slot 9A` here.

With that done, the card has a SIGN-capable, PIN-protected key on 9A and Steps 0–4 below work as written.

13.4 Prerequisites

- A card already through *Quick start: a working PIV card* (PIN set, a key + certificate in the slot you're using).
- OpenSC (`opensc-pkcs11.so`) installed on the client. Debian/Ubuntu: `apt install opensc`; Fedora: `dnf install opensc`; macOS: `brew install opensc`.
- `ssh-agent` running (`eval "$(ssh-agent)"`).
- **A contact reader** — 9A's contactless access mode is `NEVER` on both `cryptnox-default` and `ssh`, so every step below fails over a contactless-only reader, distinctly from the 9C issue above. The card behaves identically whether inserted or read by a different physical reader, as long as it's the contact interface — reader model doesn't matter, only contact vs. contactless.

13.5 Step 0 — find the PKCS#11 module path

```
$ find / -name "opensc-pkcs11.so" 2>/dev/null
```

Typical locations: `/usr/lib/x86_64-linux-gnu/opensc-pkcs11.so` (Debian/Ubuntu), `/usr/lib64/pkcs11/opensc-pkcs11.so` (Fedora), `/usr/local/lib/opensc-pkcs11.so` (Homebrew). The rest of this guide calls it `$PKCS11_MODULE`.

13.6 Step 1 — get the SSH public key off the card

```
$ ssh-keygen -D "$PKCS11_MODULE" -e
```

This lists every card object OpenSC can present as an SSH key — one line per certificate/key pair. If the card has more than one usable slot populated (e.g. both 9C and a retired key-management slot), match by fingerprint/label against `pkcs15-tool --list-certificates`.

13.7 Step 2 — authorize it

Append that public-key line to the target account's `~/.ssh/authorized_keys` — same as any other SSH key, nothing card-specific on the server side:

```
$ ssh-keygen -D "$PKCS11_MODULE" -e >> ~/.ssh/authorized_keys   # on the server, or copy␣
↪manually
```

13.8 Step 3 — load it into ssh-agent

```
$ ssh-add -s "$PKCS11_MODULE"
```

Prompts for the PIV PIN once (per `ssh-agent` lifetime, not per connection — the applet's PIN state persists until the card is removed or the session ends). `ssh-add -l` confirms the card key is loaded.

> **Note**
>
> On `cryptnox-default` this step succeeds (`Card added`) and the key *lists* in `ssh-add -l` — but see the warning at the top of this guide: the actual sign at connection time still fails for 9C. A successful `ssh-add -s` does not mean the setup will work end-to-end.

13.9 Step 4 — connect

```
$ ssh -I "$PKCS11_MODULE" user@host
```

Or make it permanent in `~/.ssh/config` so plain `ssh user@host` picks it up automatically:

```
Host host
    PKCS11Provider /usr/lib64/pkcs11/opensc-pkcs11.so
```

13.10 Troubleshooting

- **No keys listed in Step 1** — the slot has a key but no certificate (or vice versa); OpenSC needs both to expose an SSH-usable object. Check `pkcs15-tool --list-certificates` and `pkcs15-tool --list-keys`.
- **Sign operation fails / card returns 6985** — the slot's role doesn't include `SIGN`. On unmodified `cryptnox-default` that's 9A — see *Fixing it: give 9A the SIGN role*.
- **``ssh-add -s`` succeeds, PIN is correct, but connecting still fails with "Permission denied (publickey)" or "agent refused operation", and you're using 9C** — this is the `cryptnox-default`/9C issue described in *Why it fails on cryptnox-default* above, not a misconfiguration. It reproduces identically with or without `ssh-agent` in the loop, so retrying agent setup won't fix it — switch to 9A instead (*Fixing it: give 9A the SIGN role*).
- **``piv perso smoke-test`` says the PIN/PUK is blocked, but ``piv pin status`` shows full tries remaining** — you're on 9A and forgot `--slot 9A` (the command defaults to 9C); see the note in *Fixing it: give 9A the SIGN role*.

- **Login fails against a PKCS#11 tool (``CKR_USER_NOT_LOGGED_IN`` / similar) even with the correct PIN, and the reader is contactless** — 9A's contactless access mode is `NEVER`; this is the card correctly refusing the interface, not a PIN or software problem. Switch to a contact reader — see Prerequisites.

13.11 Next steps

- *Quick start: a working PIV card* — provision the key and certificate this guide builds on
- *Pre-personalization profiles* — profile YAML grammar (role, access mode, mechanisms)

CHAPTER

FOURTEEN

SSH USER CERTIFICATES

SSH public-key authentication authorizes the card's raw public key on each server's `authorized_keys`. **SSH user certificates** replace that with a trust relationship: an SSH CA signs the card's public key once, every server trusts the CA (one line in `sshd_config`), and revocation/renewal happens at the CA instead of editing `authorized_keys` everywhere the key was copied. The card's role in this is identical to the raw-key guide — it only ever signs the connection challenge; certificate issuance is a host-side step this guide covers, not a card capability.

Warning

9C cannot be used for SSH on the unmodified ``cryptnox-default`` profile, for the same reason as *SSH public-key authentication* — see *Why it fails on cryptnox-default*. The certificate layer doesn't change which key actually signs the connection; if the raw-key path fails on your card, this one will too. Use 9A instead — see *Fixing it: give 9A the SIGN role*.

Also, do not use `ssh-add` to load the certificate — it cannot load a bare certificate file (`invalid format`) when the matching private key lives on a PKCS#11 token rather than on disk. Step 4 below uses `CertificateFile` + `PKCS11Provider` instead, which is the mechanism `ssh_config(5)` documents for this case.

14.1 Prerequisites

- Complete *SSH public-key authentication* first — this guide only adds a certificate on top of that public key, it doesn't replace any of it. That includes its contact-reader requirement — 9A's contactless access mode is `NEVER`, and the certificate layer doesn't change what interface the underlying signature needs.
- An SSH CA keypair. For a self-managed CA: `ssh-keygen -t ed25519 -f ssh_ca -C "internal SSH CA"` (keep `ssh_ca` offline; `ssh_ca.pub` is what servers trust). Larger deployments typically run a CA service (e.g. Vault SSH secrets engine, `step-ca`) instead of a bare keypair — the signing command in Step 2 is the same either way, only how you invoke the signer differs.

14.2 Step 1 — export the card's public key

Same as *SSH public-key authentication* Step 1:

```
$ ssh-keygen -D "$PKCS11_MODULE" -e > card-key.pub
```

14.3 Step 2 — the CA signs it

```
$ ssh-keygen -s ssh_ca -I "user-id" -n username -V +52w card-key.pub
```

This produces `card-key-cert.pub`. `-I` is a certificate identifier (shows up in server auth logs), `-n` restricts which login principal(s) the certificate is valid for, `-V` sets an expiry — the CA can also constrain by IP, force-command, or source address; see `man ssh-keygen`. Host certificates (signing the *server's* key so clients stop trusting first-connection TOFU) use the same command with `-h`, but that's a server-key operation, unrelated to the card.

14.4 Step 3 — trust the CA on the server

One line in `/etc/ssh/sshd_config`, instead of touching `authorized_keys` per user:

```
TrustedUserCAKeys /etc/ssh/ssh_ca.pub
```

Copy `ssh_ca.pub` (never the CA private key) to the server, then `systemctl reload sshd`.

14.5 Step 4 — connect with the certificate

```
$ ssh -o PKCS11Provider="$PKCS11_MODULE" \
      -o CertificateFile=card-key-cert.pub \
      username@host
```

Do **not** use `ssh-add card-key-cert.pub` for this — `ssh-add` only loads private keys (checking file permissions as if it might be one first), and a bare certificate file has no matching on-disk private key to pair with when the key itself lives on a PKCS#11 token. `CertificateFile` is the option `ssh_config(5)` documents specifically for pairing a certificate with a key provided via `PKCS11Provider` (or `IdentityFile`/`ssh-agent` for a disk-resident key). Make this permanent in `~/.ssh/config`:

```
Host host
    PKCS11Provider /usr/lib64/pkcs11/opensc-pkcs11.so
    CertificateFile ~/card-key-cert.pub
```

14.6 Troubleshooting

- **Server still asks for a password / falls back** — confirm `TrustedUserCAKeys` points at the CA's *public* key and `sshd` was reloaded; check `sshd -T | grep trustedusercakeys`.
- **"certificate invalid: not yet valid" / "expired"** — re-sign with `-V` covering the current date; certificates don't auto-renew.
- **Principal mismatch** ("Certificate invalid: name is not a listed principal") — the `-n` value at signing time must match the login username (or use `-n` with multiple comma-separated principals).
- **"Error loading key …: invalid format" from ssh-add** — you tried `ssh-add card-key-cert.pub`, which doesn't work for a PKCS#11-resident key (see the warning at the top): use `CertificateFile` + `PKCS11Provider` in Step 4 instead.
- **"Permission denied (publickey)" / "agent refused operation" even with correct syntax and correct PIN, and you're using 9C** — this is the `cryptnox-default`/9C always-reauth issue, identical to *Why it fails on cryptnox-default*. The certificate doesn't change which key signs the connection — switch to 9A, see *Fixing it: give 9A the SIGN role*.
- Anything else from the raw-key path (module not found, other PIN/agent issues) — see *SSH public-key authentication* Troubleshooting first; this guide only adds the certificate layer on top.

14.7 Next steps

- *SSH public-key authentication* — the raw-public-key path this builds on, including the 9A fix

CHAPTER

FIFTEEN

TLS CLIENT AUTHENTICATION

TLS client (mutual) authentication proves identity to a server with a certificate instead of a password: the server requests a client certificate during the TLS handshake, and the private key signs the handshake on-card after a PIN prompt — the key never leaves the card, same as *SSH public-key authentication*.

15.1 How it has to look

Same role rule as the rest of this card: only a **SIGN-capable** slot can perform the handshake signature. On the `cryptnox-default` built-in profile that's slot **9C**; 9A is `AUTHENTICATE`-only there and cannot be used. If the card was provisioned with the **ssh** or **ms-logon** profile instead, 9A is `SIGN`-capable and works too — see *Pre-personalization profiles*. `cryptnox-default` is one named profile among several, not necessarily what any given card has — check with `factory piv preperso export-config` or whoever personalized the card.

> **Warning**
>
> Whichever slot you use, **it must be reached over the contact interface**. 9A's contactless access mode is `NEVER` on both `cryptnox-default` and `ssh` — a login attempt over a contactless reader fails at `C_Login` with `CKR_USER_NOT_LOGGED_IN`, the card correctly refusing the interface, not a bug. Use a contact reader (or a combo reader's contact slot, not its contactless pad).

The certificate's **Extended Key Usage** matters more here than for SSH: it must include *Client Authentication* (OID `1.3.6.1.5.5.7.3.2`). The self-signed certificate from *Quick start: a working PIV card* already carries it (no EKU restriction). A certificate issued from *Windows logon & Remote Desktop*'s smart-card-logon template also carries it (that template includes *Client Authentication* alongside *Smart Card Logon*), so it's reusable here — but for a server that isn't Active Directory, prefer a plain client-auth certificate without the UPN/logon-specific fields, since some relying parties reject a certificate with a Smart Card Logon EKU it doesn't expect.

15.2 Prerequisites

- A card already through *Quick start: a working PIV card* (or *Windows logon & Remote Desktop*) — a key and certificate in a SIGN-capable slot.
- A target that performs mutual TLS and is configured to trust the certificate's issuer (its own CA, or the self-signed cert imported directly into the server's trust store for testing).
- OpenSC, for the browser and command-line paths below — see *SSH public-key authentication* Step 0 for locating `opensc-pkcs11.so`.
- **A contact reader** — see the warning above; the key's contactless access mode is `NEVER`.

15.3 Firefox (NSS, cross-platform)

Firefox uses its own certificate store (NSS), independent of the OS, on every platform. Try the target site directly first:

1. Visit the target site. Some Firefox versions detect a PC/SC smart card natively and offer the card's certificate without any setup.
2. If no certificate is offered, register the module manually: `about:preferences#privacy` -> **Security Devices** -> **Load**. Module name: anything recognizable (e.g. `Cryptnox PIV`). Module filename: the `opensc-pkcs11.so` path from *SSH public-key authentication*.
3. Reload the site. On a certificate request, Firefox shows a picker listing the card's certificate; selecting it prompts for the PIV PIN once per browser session.

15.4 Windows / IIS (native store)

On Windows the inbox PIV minidriver exposes the card to CryptoAPI/CNG, so any application using the Windows certificate store (IIS, Edge, curl built against Schannel) sees the card certificate without a PKCS#11 module, the same mechanism *Windows logon & Remote Desktop* relies on for logon. Configure IIS to *request* or *require* client certificates on the site binding, which is a server-side setting rather than a card-side one, and the handshake triggers the PIN prompt.

Three card-side requirements apply here specifically:

- **9A must be SIGN-capable**, so the card needs the `ms-logon` (or `ssh`) profile. Windows performs client authentication with the 9A key, and `cryptnox-default`'s 9A is `AUTHENTICATE`-only, which cannot produce the handshake signature.
- **The 9A key must be RSA.** The inbox minidriver does not enumerate EC keys: with an EC key the card produces no key container at all, so no certificate reaches the Windows store and nothing can be offered in the picker. This is a Windows driver property rather than a card one — it reproduces on a YubiKey, and P-384 behaves like P-256. Pass `--algorithm RSA2048` to `piv quickstart --profile ms-logon --slot 9A` for this reason (the ECC default warns on that combination). See *Windows logon & Remote Desktop* for the detail.
- **The card must not carry a Discovery Object container.** No built-in profile creates one, so this only affects cards where `oid: 7E` was added by hand. Such a card is rejected outright by the minidriver: nothing enumerates and `certutil -scinfo` reports `NTE_BAD_KEYSET`. See *Pre-personalization profiles*.

Warning

Three server-side settings are easy to get wrong, and each produces a failure that looks like a card fault:

- **``appcmd`` needs ``/commit:apphost``.** `system.webServer/security/access` is locked at the global level, so without it the write lands in the site's `web.config` and is refused. `appcmd` is a native executable, so this does not raise in PowerShell. `sslFlags` silently stays `None`, IIS never asks for a client certificate, and the browser test yields a false negative. Always read the value back with `Get-WebConfigurationProperty`.
- **TLS 1.3 breaks browser client-certificate auth on Windows 11 before 24H2.** TLS 1.3 replaced renegotiation with post-handshake authentication, which Chrome and Edge do not implement and Firefox does not enable by default, so http.sys resets the connection (`ERR_CONNECTION_RESET` / `PR_CONNECT_RESET_ERROR`). Enable negotiation on the binding instead of disabling TLS 1.3 machine-wide:

  ```
  netsh http add sslcert ipport=0.0.0.0:443 certhash=<thumbprint> \
      appid=<appid> certstorename=MY clientcertnegotiation=enable
  ```

  Confirm with `netsh http show sslcert`; the line that matters is `Negotiate Client Certificate : Enabled`.

- **A successful ``curl`` does not prove the browser path works.** Windows `curl` uses Schannel, which does support post-handshake authentication, and ships without HTTP/2, so it returns `200 OK` in exactly the configuration where every browser fails.

Note

In *request* mode (`SslNegotiateCert`) the page still loads if you cancel the certificate picker, so a page load alone does not prove the card signed anything. To prove it, use *require* mode (`SslRequireCert`): a client with no certificate is then refused with `403.7` on the same binding. Close every browser window between runs, since both the certificate choice and the TLS session are cached.

Also type `https://` in full. Edge resolves a bare `localhost` to `http://`, which returns `403.4 ... secured with SSL` from port 80 without ever touching the HTTPS binding.

15.5 curl / OpenSSL (scripted clients)

The exact mechanism depends on your curl/OpenSSL build. Check `curl -V` for the SSL backend, then `curl --engine list` — most current builds (OpenSSL 3.x with `ENGINE` support removed) have no PKCS#11 engine compiled in, in which case use the provider-based path below instead of the older `pkcs11:` URI + legacy engine approach some older guides show.

OpenSSL 3 provider path (needs the `pkcs11-provider` package — `dnf install pkcs11-provider` on Fedora, or your distro's equivalent):

```
# openssl.cnf (a dedicated file via OPENSSL_CONF, not necessarily the
# system config - activates both the default and pkcs11 providers)
openssl_conf = openssl_init
[openssl_init]
providers = provider_sect
[provider_sect]
default = default_sect
pkcs11 = pkcs11_sect
[default_sect]
activate = 1
[pkcs11_sect]
activate = 1
pkcs11-module-path = /usr/lib64/pkcs11/opensc-pkcs11.so
```

```
$ export OPENSSL_CONF=/path/to/that/openssl.cnf
$ curl --cacert server-ca.pem \
       --cert 9c.crt.pem \
       --key 'pkcs11:serial=<card serial>;type=private' --key-type PROV \
       https://target.example.com
```

Two details that aren't obvious from the `pkcs11:` URI syntax alone:

- **The certificate must come from a plain file, not a ``pkcs11:`` URI.** `pkcs11-provider` implements PKCS#11 key objects only, no `CKO_CERTIFICATE` support (its docs and man page never mention certificates). A `type=cert` URI silently matches nothing; it isn't an error, it just won't work. Export the certificate once (*Quick start: a working PIV card* already does, as `9c.crt.pem`/`9a.crt.pem`) and point `--cert` at that file.
- **Plain ``pkcs11:`` URIs need ``–cert-type PROV``/``–key-type PROV``.** Without it, curl tries the legacy

OpenSSL `ENGINE` API and fails with `crypto engine not set, cannot load certificate` on any build without a pkcs11 engine compiled in (`curl --engine list` shows what's available — most current builds have none).

- **``serial=`` is the reliable URI identifier** if more than one PKCS#11 token could be present (e.g. a second reader, or a token-less empty slot). A bare `token=<label>` or unqualified `pkcs11:` URI may resolve to the wrong slot, or hang waiting for a PIN on a reader you didn't mean to use.

15.6 Troubleshooting

- **Handshake fails / card returns 6985** — the slot's role doesn't include `SIGN`; see *How it has to look* above.
- **Server rejects the certificate** — check the EKU includes *Client Authentication*, and that the server trusts the issuing CA (or the self-signed cert is imported into its trust store directly).
- **No certificate offered in the picker** — the store (NSS/CryptoAPI) isn't seeing the card; confirm the PKCS#11 module loaded (Firefox) or that the minidriver recognizes the card (Windows: `certutil -scinfo`).
- **Windows: ``NTE_BAD_KEYSET`` from ``certutil -scinfo``, nothing enumerates** — the minidriver found no containers on the card. Check the two card-side requirements in *Windows / IIS (native store)* above: a SIGN-capable 9A, and no Discovery Object container. Note this error is not specific; an entirely blank PIV applet produces it too.
- **Windows: certificates appear in the picker that don't belong to this card** — certificates propagate into `Cert:\CurrentUser\My` and *stay there after the card is removed*. A stale entry makes the picker look correct while the inserted card contributes nothing. Match on the certificate subject before concluding the card was read.
- **curl: "No cert found in the openssl store" with no other error** — you used a `pkcs11:` URI for `--cert`; switch to a plain certificate file (see the notes above — the provider doesn't support certificate objects).
- **curl hangs, or prompts for a PIN on a reader/token you didn't expect** — more than one PKCS#11 token is visible (e.g. a second reader plugged in); add `serial=<card serial>` to the URI to disambiguate instead of a bare `token=` label.
- **Login fails (``CKR_USER_NOT_LOGGED_IN`` or similar) with the correct PIN over a contactless reader** — 9A/9C's contactless access mode is `NEVER`; this is the card refusing the interface by design, not a bug. Switch to a contact reader.

15.7 Next steps

- *Quick start: a working PIV card* — provision the key and certificate this guide builds on
- *SSH public-key authentication* — the same PKCS#11 path, for SSH instead of TLS
- *Pre-personalization profiles* — the `ssh/ms-logon` profiles' 9A `SIGN` role

CHAPTER

SIXTEEN

MACOS SMART-CARD LOGIN

macOS recognizes a PIV card for login through Apple's own **CryptoTokenKit** `pivtoken` driver, not OpenSC/PKCS#11, there is no bypass. Pairing a card to a local account is additive by default: the card becomes an alternative to your password at the lock screen, `sudo`, and full login, not a replacement. Requiring the card and removing password login entirely is a separate, optional, higher-risk step covered at the end.

Warning

macOS's bundled CCID driver (still v1.5.1 as of macOS Tahoe) fails to negotiate this card's ATR outright: connect attempts return "unresponsive," even though the ATR itself reads fine. This isn't a reader problem (two different reader models fail identically) or a dead card (a T=0 connect attempt gets `E_PROTO_MISMATCH` rather than "unresponsive," proving the reader is talking to the card, just failing to open the T=1 session the card requires). The fix is Step 0 below; without it, nothing else in this guide will work.

16.1 How it has to look

Same role rule as the rest of this card: the login/authentication signature needs a **SIGN-capable** slot. This guide is written and tested against **9A**, provisioned with the **ssh** profile (see *Pre-personalization profiles*) — the same key used in *SSH public-key authentication*/*TLS client authentication*.

Note

9C is **not the first choice here**: it pairs and unlocks the screen, but full login with it has not been tested. 9C carries the PIV spec's `nonRepudiation`/always-reauth convention (confirmed elsewhere on this card: a fresh re-login is required before every single signature, which breaks SSH/TLS via OpenSC — see *SSH public-key authentication*'s *Why it fails on cryptnox-default*). Use a 9A that can sign; on a `cryptnox-default` card, see the next section.

`cryptnox-default` is one named profile among several, not necessarily what any given card has — check with `factory piv preperso export-config` or whoever personalized the card. Unlike Windows AD, macOS local pairing doesn't require a specific EKU or UPN in the certificate — a plain client certificate from *Quick start: a working PIV card* is enough.

16.2 Cards on the cryptnox-default profile

On `cryptnox-default` the 9A key can authenticate but not sign. Pairing the 9A identity then fails: `sc_auth pair` stops with CryptoTokenKit error -3, and the CryptoTokenKit log shows "SmartCard returned error 6985". `cryptnox-id piv inventory` lists the role of every key object; a 9A with only `AUTHENTICATE` cannot be used here. A key's role cannot be changed, so there are two ways on:

- **Use 9C.** The 9C identity pairs and unlocks the screen, asking for the PIN at each signature, which is the 9C convention. Full login with 9C has not been tested.
- **Add a signing key object to 9A.** A slot holds one key object per algorithm, so add one for an algorithm 9A does not have yet, with the SIGN role, then generate its key and certificate:

```
$ cryptnox-id piv perso create-key-object --slot 9A --algorithm RSA2048 \
    --role SIGN+AUTHENTICATE --default-keys
$ cryptnox-id piv perso generate-key --slot 9A --algorithm RSA2048 \
    --out 9a.pub.pem --default-keys
$ cryptnox-id piv perso self-sign-cert --slot 9A --algorithm RSA2048 \
    --public-key 9a.pub.pem --subject "CN=Mac Login" --out 9a.pem
$ cryptnox-id piv perso import-cert --slot 9A --cert 9a.pem --default-keys
$ cryptnox-id piv perso smoke-test --slot 9A --algorithm RSA2048
```

  The key object is permanent, and a finalized card refuses it. The old 9A key stays on the card, unused; the new certificate replaces the old one. Remove the old pairing with `sc_auth unpair`, then pair the new identity as in Step 1. `ECCP384` works the same way in place of `RSA2048`. Pairing with an RSA-2048 or P-384 9A has not been tested on a Mac.

16.3 Prerequisites

- A card already through *Quick start: a working PIV card* (key + certificate in a SIGN-capable slot), with the standard PIV objects written (`write-standard-objects` — CryptoTokenKit identifies the card as PIV through the CCC/CHUID).
- Admin access on the Mac.
- OpenSC, for the diagnostic check in Step 0 (`brew install opensc`). The actual login path doesn't use it, or need `cryptnox-id` installed on the Mac at all — GUI login goes through Apple's own `pivtoken` driver exclusively.
- A **contact** reader. Only contact readers have been tried for this guide — by analogy with SSH/TLS (same signing key, same access-mode restriction) contactless is expected not to work, but that hasn't been separately confirmed on macOS.

16.4 Step 0 — fix macOS's CCID driver

```
$ sudo defaults write /Library/Preferences/com.apple.security.smartcard useIFDCCID -bool␣
↪yes
```

Then physically **unplug and replug the reader**, with the card inserted, so the smartcard service reloads the driver. This switches macOS to the alternative CCID driver (the actively maintained upstream one) instead of Apple's outdated bundled one — not Cryptnox-specific; the CCID driver's own author recommends this as the first troubleshooting step for any macOS smartcard problem. The setting persists across reboots; if a later macOS update breaks smart-card login, check this first.

Confirm the card now responds:

```
$ opensc-tool -n
```

Should report the card as a "Personal Identity Verification Card." If it still says "unresponsive," the driver switch didn't take, confirm the `defaults write` succeeded and that you actually unplugged and replugged the reader afterward (a service restart alone isn't enough).

16.5 Step 1 — pair the certificate to the account

Find the card's identity hash:

```
$ sc_auth identities
```

Shows an "Unpaired identities" line with a hash and the certificate's CN. Pair it:

```
$ sc_auth pair -u $(whoami) -h <hash-from-above>
```

Prompts for your Mac account password once (to authorize the pairing), then the card's PIV PIN. `sc_auth list -u $(whoami)` confirms the pairing afterward. This is additive — your password still works everywhere; the card is now an alternative, not a replacement.

> **Note**
>
> `security find-identity -p smartcard` does not work on current macOS — the `smartcard` policy value isn't supported by `find-identity`. Use `sc_auth identities` instead.

16.6 Step 2 — use it

- **Lock screen**: lock the screen with the card inserted, enter the PIV PIN instead of your password to unlock.
- `sudo`: `sudo -v` in Terminal accepts the card PIN too — macOS wires the card into `sudo`'s PAM stack automatically, no separate setup.
- **Full login**: log out, sign in with the card inserted using the PIN.

16.7 Requiring the card (optional, higher risk)

> **Warning**
>
> Unlike pairing, this removes the password fallback entirely. Test pairing thoroughly first (Step 2 above) with a card and reader you trust, before disabling password login. Keep an active admin session or physical access available until you've confirmed the card works at the login window with enforcement on.

```
$ sudo sysadminctl -smartcardstatus enable
```

Requires a smart card for **all** local accounts, not just the paired one. For managed fleets, Apple's current guidance is a Smart Card configuration profile pushed via MDM instead of this local command.

For pre-boot (FileVault) card requirement specifically, `sc_auth filevault -o enable` exists but carries the same class of risk at the very first screen of a cold boot, before any recovery options are available — leave it off unless you specifically need it.

16.8 Troubleshooting

- **PIN entry stops working, card seems locked** — too many wrong PIN attempts blocks the card's PIN (6 tries on the default profile, see *Quick start: a working PIV card*); unblock with the PUK (`sc_auth changepin`, or from the Linux/CLI side). This is a card PIN lockout, not a login lockout — your password still works unless you've also enabled "require card" above.

- **``sc_auth: command not found`` / no ``-smartcardstatus`` flag** — Apple has changed and relocated these commands across macOS releases; check `man sc_auth` and `man sysadminctl` on the target OS, or use an MDM Smart Card profile instead.
- **Card not offered at login, or "unresponsive card" from any tool** — confirm Step 0's driver fix is applied and the reader was replugged afterward; this is the most common cause. Then confirm `opensc-tool -n` sees the card outside the login window.
- **Locked out after enabling "require card"** — boot into Recovery / Safe Mode, or use another admin account, to run `sudo sysadminctl -smartcardstatus disable`.
- **Cert expired** — login (and pairing) stops working once the PIV certificate expires; issue a new one and re-pair.
- **Card only responds over a contactless/NFC-capable reader, or a combo reader's contactless pad** — this card's signing slot has its contactless access mode set to `NEVER`; the card itself refuses the private-key operation login needs over that interface, by design, not a driver or macOS bug. Use the reader's contact slot instead. The exact macOS-side error text for this case hasn't been captured yet (not independently tested over NFC on a Mac); on Linux/OpenSC the same underlying refusal surfaces as `CKR_USER_NOT_LOGGED_IN` at login, over a contact reader it works normally.

16.9 Next steps

- *Quick start: a working PIV card* — provision the key and certificate this guide builds on
- *Pre-personalization profiles* — profile options for the signing slot

CHAPTER

SEVENTEEN

PIV PERSONALIZATION

The full PIV lifecycle: pre-personalization (structure), PIN/PUK values, on-card key generation, external key import, CSRs and certificates, standard data objects, validation, and the irreversible finalize step.

For the condensed version, see the *Quick start: a working PIV card*.

> **Note**
>
> This card provisions the PIV **Application PIN** (key ref `0x80`) only — there is no Global PIN option, and PIV's PIN is independent from FIDO2's (DESFire has no PIN at all; its access is symmetric-key based). Each application's PIN has its own retry counter and is managed separately.

Operator commands live under `piv`; factory pre-personalization lives under `factory piv preperso`. The SCP03 admin channel uses the keys of the card's PIV security domain — as shipped, the GlobalPlatform test keys (`--default-keys`); after *Rotating the PIV admin keys*, the `PIV_SCP03_ENC` / `PIV_SCP03_MAC` / `PIV_SCP03_DEK` environment variables. `PIV_MGMT_KEY` carries the PIV management key (`9B`) for cards that need it (see Step 2).

17.1 Quickstart (one shot)

`piv quickstart` chains the steps below over one card connection, skipping whatever is already done (re-runs converge), and ends with the `yubico-piv-tool` commands that verify the result. It detects the card state first; `--dry-run` shows the step plan without writing.

```
$ cryptnox-id piv quickstart --default-keys --dry-run
$ cryptnox-id piv quickstart --default-keys                          # 9C, ECCP256, self-
↪signed cert
$ cryptnox-id piv quickstart --default-keys --cert-mode csr --csr-out 9c.csr.pem
$ cryptnox-id piv quickstart --default-keys --include-preperso      # blank card: lay␣
↪down the structure too
```

> **Note**
>
> Secrets are resolved up front (`CRYPTNOX_PIV_NEW_PIN` / `CRYPTNOX_PIV_NEW_PUK` or masked prompts; `CRYPTNOX_PIV_PIN` when the PIN already exists). Quickstart defaults to ECC P-256, except with `--profile ms-logon` on slot 9A, the Windows shape, where it defaults to RSA-2048 because Windows does not enumerate EC keys. For RSA on any other card shape, use `generate-key` directly on a slot with an RSA key object, or *import a key*. A slot that already has a certificate is left untouched (never silently replaces a certified key); with `--cert-mode csr` a re-run regenerates the key and invalidates a previously exported CSR. `finalize` is never invoked by quickstart. The first failure stops the run with partial results and exit code 6; completed steps are kept.

The same sequence, below, as individual commands — useful to understand what quickstart does, or to vary it.

17.2 Step 0 — factory: pre-personalize the applet structure

> **Note**
>
> **Factory / provisioning step — not part of normal end-user setup.** Pre-personalization is done once at manufacturing; a card you receive is already structured, so start at Step 1. Full detail lives under *Factory commands*.

`load-config` applies a profile (containers, PIN/PUK verifiers, key slots) to a blank OpenFIPS201 applet over SCP03 — one short `PUT DATA ADMIN` per command. See *Pre-personalization profiles* for the profile format itself.

```
$ cryptnox-id factory piv preperso status
$ cryptnox-id factory piv preperso inspect-defaults
$ cryptnox-id factory piv preperso load-config --profile cryptnox-default --dry-run
$ cryptnox-id factory piv preperso load-config --profile cryptnox-default --default-keys
```

17.3 Step 1 — set the PIN and PUK

```
$ cryptnox-id piv perso set-puk --default-keys     # prompts (masked) for the new PUK
$ cryptnox-id piv perso set-pin --default-keys     # prompts (masked) for the new PIN
$ cryptnox-id piv pin status                       # configured? retries?
```

17.4 Step 2 — generate keys on the card

Keys are generated **on-card**; only the public key leaves. Use a SIGN-role slot (9C) for anything you will sign with.

```
$ cryptnox-id piv perso generate-key --slot 9C --algorithm ECCP256 --out 9c.pub.pem --
↪default-keys
$ cryptnox-id piv perso generate-key --slot 9A --algorithm ECCP256 --out 9a.pub.pem --
↪default-keys
```

A slot generates on-card for any mechanism it has a **key object** for, and the pre-personalization profile decides which those are (see *Pre-personalization profiles*). Every key object in `cryptnox-default` is ECC (`ECCP256`, `ECCP384`), so on such a card RSA has to be imported with `import-key` (next). A profile that provides an RSA object generates RSA on-card just as well — `ms-logon` creates an `RSA2048` object on 9A, and:

```
$ cryptnox-id piv perso generate-key --slot 9A --algorithm RSA2048 --out 9a.pub.pem --
↪default-keys
```

succeeds there, with the private key never leaving the card. Requesting a mechanism the slot has no object for returns `6A80`.

Over the admin channel some cards deliver at most 256 bytes of a public-key template, which an RSA template exceeds. `generate-key` reads that from the response and completes the generation with the PIV management key (`9B`) over plain APDUs; the command line is the same either way. That path is contact-only and needs the `9B` value — `--default-keys` on development cards, `PIV_MGMT_KEY` otherwise. Profiles create the `9B` key object but carry no value; load one once with:

```
$ cryptnox-id factory piv preperso set-mgmt-key --default-keys
$ cryptnox-id piv perso generate-key --slot 9A --algorithm RSA2048 --out 9a.pub.pem --
↪default-keys
```

`piv quickstart` follows the same rule with training wheels: it defaults to ECC P-256 everywhere, accepts explicit RSA only for `--profile ms-logon` on 9A (the one built-in shape with an RSA object, and the one Windows needs — the ECC default warns there), and elsewhere refuses RSA early rather than failing on the card. RSA-1024 is removed from the build; the CLI never offers it.

17.5 Step 2b — import an external private key

`import-key` injects a host-generated key over SCP03 — for keys that must be generated off-card (CA/HSM-issued, escrow or migration scenarios); for on-card RSA see Step 2, which works on any slot with an RSA key object. The key travels as CHANGE REFERENCE DATA ADMIN elements, **one element per SCP03 session** (JCOP 4.5), with large elements ISO-chained; the sequence starts with a CLEAR, so re-running an import is safe. Accepts PEM/DER, PKCS#8 or traditional, encrypted or not (`CRYPTNOX_PIV_KEY_PASSWORD` env var, or a masked prompt).

```
$ openssl genrsa -out rsa2048.key.pem 2048
$ cryptnox-id piv perso import-key --slot 9C --key rsa2048.key.pem --default-keys --dry-
↪run
$ cryptnox-id piv perso import-key --slot 9C --key rsa2048.key.pem --default-keys \
    --create-key-object --public-out 9c-rsa.pub.pem
```

The target key **object** must exist with the same (slot, mechanism) pair and the IMPORTABLE attribute. The `cryptnox-default` profile creates ECC-P256 objects only, so an RSA (or P-384) import needs either a profile that defines it (the `ms-logon` profile pre-creates an importable RSA-2048 object on 9A — see *Pre-personalization profiles*) or `--create-key-object` (a structural `PUT DATA ADMIN`: development/evaluation cards only — a finalized applet refuses it). RSA objects fix CRT vs. plain form at creation; `--rsa-form` must match, or the card rejects the import (`6A80`). A post-import smoke test signs on-card and verifies against the imported key (skip with `--no-smoke-test`; a non-SIGN slot reports "cannot sign" — that is the slot role, not a failure).

For a PKI-issued credential delivered as PKCS#12, `import-p12` runs the key import **and** the certificate import in one command (the Windows smart-card logon / Remote Desktop enrollment path — see *Windows logon & Remote Desktop*):

```
$ cryptnox-id piv perso import-p12 --slot 9A --p12 user.pfx --default-keys
```

17.6 Step 3 — CSR and certificates (on-card signing)

```
$ cryptnox-id piv perso generate-csr   --slot 9C --subject "CN=Test User" \
    --public-key 9c.pub.pem --out 9c.csr.pem
$ cryptnox-id piv perso self-sign-cert --slot 9C --subject "CN=Test User" \
    --public-key 9c.pub.pem --out 9c.crt.pem
$ cryptnox-id piv perso import-cert    --slot 9C --cert 9c.crt.pem --csr 9c.csr.pem
```

Signing requires a SIGN-role slot; an AUTHENTICATE-only slot (e.g. 9A) returns `6985`. Large certificates are written with ISO command chaining automatically. `import-cert` writes the certificate it is given; `--csr` refuses one whose public key is not the CSR's, and `--verify-pin` has the slot sign a challenge and refuses unless the certificate's key verifies it.

17.7 Step 4 — data objects

```
$ cryptnox-id piv perso generate-chuid
$ cryptnox-id piv perso generate-ccc
$ cryptnox-id piv perso generate-discovery
$ cryptnox-id piv perso write-standard-objects --default-keys
$ cryptnox-id piv objects list
```

`generate-chuid`/`generate-ccc`/`generate-discovery` build the object bytes locally (no card, no keys needed) — pipe them into `write-object`, or use `write-standard-objects` to generate and write CHUID + CCC in one step.

17.8 Step 5 — validate and smoke-test

```
$ cryptnox-id piv perso smoke-test       # verify PIN, sign with 9C, read objects/certs
$ cryptnox-id piv validate               # consistency check (NOT a FIPS/NIST validation)
$ cryptnox-id report piv --out piv.json
```

17.9 Step 6 — finalize (irreversible, factory)

> **Note**
>
> **Factory / provisioning step — not part of normal end-user setup.** Finalizing is a manufacturing operation and is irreversible. See *Factory commands*.

`finalize` transitions the applet to its `SECURED` operational state. It is irreversible; a full reset of the PIV applet runs through the Cryptnox remote PIV service (*Remote service guide*). Interactively it asks you to type the token `FINALIZE-PIV`; non-interactively it requires the `--i-understand-this-is-irreversible` flag instead — either satisfies the gate.

`finalize` locks the **structure** only: which containers, verifiers and key objects exist, and their access rules. Administration with the security domain keys continues afterwards — key generation and import, certificate import, setting a new PIN or PUK, and rotating the admin keys themselves. Rotate the admin keys *before* finalizing a production card (*Rotating the PIV admin keys*), so that finalize already runs with your production keys.

```
$ cryptnox-id factory piv preperso finalize --default-keys
# at the prompt: Type FINALIZE-PIV to continue
# (scripted/non-interactive: add --i-understand-this-is-irreversible)
```

17.10 Rotating the PIV admin keys

Every card ships with the published GlobalPlatform test key on **key version 1** of its PIV security domain (AID `A0000001515350410`1), so that the first personalization works out of the box. Replace it before a card goes into production. The key set is three AES-128 keys, key identifiers 1 = ENC, 2 = MAC, 3 = DEK.

Key version 2 of the same domain holds a per-card Cryptnox key set. It gives the same PIV administration rights and is the recovery path if your own keys are lost. Leave it in place (see *Deleting key version 2* below). The card manager (ISD) is a separate domain and is not involved.

The security domain keys *are* PIV administration: whoever holds them can generate or replace keys, import certificates and set a new PIN or PUK without knowing the current one, before and after `finalize`. Protect them like CA issuing keys, and use one key set per card.

The rotation is a single GlobalPlatform PUT KEY that replaces key version 1 in place. `piv admin rotate-keys` sends it; the equivalent GlobalPlatformPro (`gp`, v25.10.20 or later) lines are given for sites that script with it. Card on a contact reader. The CLI keeps finding your keys afterwards without any configuration beyond the environment variables.

1. Read the key table. No key is needed and nothing is counted against the card:

   ```
   $ cryptnox-id piv admin status
   ```

   Expect key versions 1 and 2, three AES-128 keys each. The replacement key set takes the type the domain speaks, AES here: a domain is bound to SCP02 or SCP03 when it is created, and a key set of the other type is accepted by the card yet can never open a session afterwards, which only the card manager can undo by deleting and re-creating the domain. `rotate-keys` writes the matching type; some other tools send DES keys unless told otherwise. `piv admin keys --check-default` tells, with one authentication attempt, whether version 1 still answers to the test key.

   The same read with raw commands, for other tools:

   ```
   SELECT    00 A4 04 00 09 A0 00 00 01 51 53 50 41 01
   GET DATA  80 CA 00 E0 00
   ```

2. Replace key version 1 in place, keeping version number 1. Generate three independent random 16-byte keys first (in an HSM or key vault); the CLI takes them from the environment, never from the command line:

   ```
   $ export PIV_SCP03_NEW_ENC=<ENC> PIV_SCP03_NEW_MAC=<MAC> PIV_SCP03_NEW_DEK=<DEK>
   $ cryptnox-id piv admin rotate-keys --default-keys --dry-run
   $ cryptnox-id piv admin rotate-keys --default-keys
   ```

   The command reads the key table, writes only version 1, compares the key check values the card echoes with its own, authenticates once with the new keys and re-reads the table to confirm version 2 is as it was. With GlobalPlatformPro instead:

   ```
   $ gp -c A00000015153504101 --key 404142434445464748494A4B4C4D4E4F --key-ver 1 \
        --lock-enc <ENC> --lock-mac <MAC> --lock-dek <DEK> --new-keyver 1
   ```

   Warning

   Never pass `--new-keyver 2` to `gp`. The card accepts it and silently overwrites the Cryptnox key set in version 2, while version 1 stays on the test key; the key table looks the same afterwards. Omitting `--new-keyver` is also correct, `gp` then replaces version 1. `rotate-keys` never addresses version 2.

3. Switch the CLI to the new keys and stop passing `--default-keys`. The successful authentication and the key table are the proof of the rotation:

   ```
   $ export PIV_SCP03_ENC=<ENC> PIV_SCP03_MAC=<MAC> PIV_SCP03_DEK=<DEK>
   $ cryptnox-id piv admin authenticate
   $ cryptnox-id piv admin keys
   ```

   With GlobalPlatformPro, `gp -c A00000015153504101 --key-enc <ENC> --key-mac <MAC> --key-dek <DEK> --key-ver 1 -i` does the same in a new session.

Do not try the old keys to confirm the rotation. The card counts every authentication that does not end in a successful EXTERNAL AUTHENTICATE, including attempts a host abandons after a card-cryptogram mismatch. After five consecutive ones, every INITIALIZE UPDATE on the card, in any security domain, is delayed by about 24 seconds until the next successful authentication. There is no lockout, only the delay.

A PUT KEY with a malformed key block, a wrong key check value or keys wrapped with the wrong DEK is refused (`6982`) and the key table is unchanged; `rotate-keys` stops there and the current keys still work. If a rotation is interrupted after the card accepted the PUT KEY, the card expects the new keys: run `piv admin authenticate` with the new values as `PIV_SCP03_*`. Should neither key set authenticate, find out which MAC key the card holds (INITIALIZE UPDATE with key version 1, then check the card cryptogram on the host with the new MAC key and with the old one), and re-run step 2 authenticated with that key set.

17.10.1 Deleting key version 2

If your policy requires sole custody, delete the Cryptnox key set after rotating version 1, authenticated with your new keys:

```
$ cryptnox-id piv admin delete-factory-keyset
# at the prompt: Type DELETE-FACTORY-KEYSET to continue
# (scripted/non-interactive: add --i-understand-this-is-irreversible)
```

The command refuses while the `PIV_SCP03_*` variables still hold the test key, and re-reads the key table afterwards. With GlobalPlatformPro: `gp -c A00000015153504101 --key-enc <ENC> --key-mac <MAC> --key-dek <DEK> --key-ver 1 --delete-key 2`.

This cannot be undone. Afterwards Cryptnox cannot recover PIV administration on that card if your keys are lost; the only remaining path is a full reset of the PIV applet, which erases its contents (a Cryptnox operation — contact Cryptnox support).

17.11 Next steps

- *Pre-personalization profiles* — customizing the pre-personalization profile
- *Quick start: a working PIV card* — the condensed, one-command path with a `yubico-piv-tool` interoperability matrix
- *PIV commands* — every `piv` command

CHAPTER

EIGHTEEN

PIV COMMANDS

Operator and personalization commands for the PIV function (SP 800-73). Administration runs over an SCP03 secure channel and works over both interfaces (the PIN, the PUK and the PIN-protected keys are **contact-only**); admin keys come from `--default-keys` (the test keys every card ships with) or, once replaced (*Rotating the PIV admin keys*), the `PIV_SCP03_ENC` / `PIV_SCP03_MAC` / `PIV_SCP03_DEK` environment variables. PIN values come from masked prompts or `CRYPTNOX_PIV_PIN` / `CRYPTNOX_PIV_NEW_PIN` / `CRYPTNOX_PIV_NEW_PUK` — never the command line.

On-card generation of a large public key can also use the card's **PIV management key** (key reference `9B`), whose value comes from `--default-keys` or `PIV_MGMT_KEY` — never the command line. `--default-keys` covers both the admin channel and the management key: when it is given, `PIV_MGMT_KEY` is not read.

18.1 Inspection

```
piv info            applet AID/label, supported algorithms, key slots
piv status          lifecycle state, PIN/PUK status, object presence
piv discover        the Discovery object (PIN usage policy)
piv slots           key slots and which hold a certificate
piv inventory       key objects, verifiers, containers and config, with access rules
piv validate        consistency check (NOT a NIST/FIPS validation)
piv quickstart      one-shot personalization (see the PIV quick start)
```

`piv inventory` is the applet's own listing of its structure, read without authentication: every key object with its mechanism, roles, attributes and access modes, every verifier with its length and retry limits, every container with its access modes, plus the applet version, its finalize state and its config flags. An applet whose `restrict-enumeration` flag is set withholds the structure (`6986`) and the command says so; version, state and config still answer.

18.2 PIN

```
piv pin status      PIN/PUK configured? retries left? (non-decrementing)
piv pin verify      verify the PIN (a wrong PIN consumes one retry)
piv pin change      change the PIN (cardholder; needs the current PIN)
piv pin unblock     set a new PIN with the PUK (RESET RETRY COUNTER; works on
                    a PIN that is not blocked too; a wrong PUK consumes a PUK
                    retry, see warning below)
```

18.3 PUK

```
piv puk change      change the PUK (cardholder; needs the current PUK)
```

`piv pin change` and `piv puk change` are cardholder operations — they prove knowledge of the current value in-band (no SCP03 admin channel), unlike the `piv perso set-pin` / `set-puk` issuance commands. `piv pin unblock` sets a new PIN and resets its retry counter using the PUK. It works on a PIN that is not blocked as well, which makes it a test that the PUK is right, and a correct PUK also resets the PUK's own retry counter. Current/new values come from masked prompts or the `CRYPTNOX_PIV_PIN` / `CRYPTNOX_PIV_NEW_PIN` / `CRYPTNOX_PIV_PUK` / `CRYPTNOX_PIV_NEW_PUK` environment variables — never the command line.

> **Warning**
>
> The PUK has its own retry counter, and every wrong PUK — entered through `piv pin unblock`, `piv puk change`, or any other tool — consumes one retry. Exhausting the PUK retries is **permanent**: a blocked PUK has no recovery path. A full reset of the PIV applet, which erases all keys and certificates, runs through the Cryptnox remote PIV service (*Remote service guide*). Check `piv pin status` (non-decrementing) before guessing.

18.4 Admin channel

```
piv admin status        the admin security domain: SCP version, key versions (no auth)
piv admin keys          its key table; --check-default tests key version 1 (one attempt)
piv admin authenticate  open the channel (mutual auth) + harmless self-test
piv admin rotate-keys   replace key version 1 in place (new keys from PIV_SCP03_NEW_*)
piv admin delete-factory-keyset
                        IRREVERSIBLY delete key version 2, Cryptnox's keyset
```

`piv admin status` selects the PIV security domain and reads its key information template. It never sends INITIALIZE UPDATE: on this chip an INITIALIZE UPDATE that is not followed by a successful EXTERNAL AUTHENTICATE counts as a failed authentication, and a status read must not spend one.

`piv admin keys` lists the same table. With `--check-default` it authenticates once to key version 1 with the publicly known GlobalPlatform test key: a success means anyone can administer the PIV applet, and leaves the card's failed-authentication count clear; a wrong key costs one failed authentication, which the next successful `piv admin authenticate` clears. It never tries more than that one key, and skips the check when the table has no key version 1.

`piv admin rotate-keys` replaces key version 1 of the PIV security domain in place: one PUT KEY carrying ENC, MAC and DEK, encrypted under the current DEK, with the key version byte left at 1. The current keys come from `--default-keys` or `PIV_SCP03_ENC` / `PIV_SCP03_MAC` / `PIV_SCP03_DEK`; the new ones from `PIV_SCP03_NEW_ENC` / `PIV_SCP03_NEW_MAC` / `PIV_SCP03_NEW_DEK` (hex, 16 bytes each), never the command line. The command reads the key table first, refuses to write anything but key version 1, compares the key check values the card echoes with its own, authenticates once with the new keys, and re-reads the table to confirm key version 2 is as it was. Key version 2 is Cryptnox's keyset for remote reset and attestation; the command never addresses it. `--dry-run` prints the plan with the new keys' check values and sends nothing. A refused PUT KEY writes nothing, so the current keys still work. The new keys take the type the domain speaks, DES on SCP02 and AES on SCP03: a domain is bound to one of the two when it is created, and a keyset of the other type is accepted by the card yet can never open a session afterwards.

`piv admin delete-factory-keyset` deletes key version 2 (DELETE KEY by key version), authenticated with key version 1 from the `PIV_SCP03_*` variables. It refuses while those are the publicly known test keys, and needs the typed confirmation `DELETE-FACTORY-KEYSET` or `--i-understand-this-is-irreversible`. Afterwards Cryptnox's remote reset and attestation cannot reach the card; key version 1 is the only way left to administer the PIV applet.

18.5 Certificates and objects

```
piv certs list          certificate slots, subjects, expiry
piv certs export        export a certificate (PEM/DER) - certificates are not secret
piv certs inspect       certificate details for a slot
piv objects list        PIV data objects and presence
piv objects read        read a data object (hex or --out FILE)
```

18.6 Attestation

```
piv export-attestation  export the on-card key-attestation leaf (PEM chain / DER)
piv verify-attestation  validate the attestation chain to the pinned Cryptnox root
```

The factory/issuance-time key-attestation leaf (slot 9C in this version) is stored in the retired-slot-95 certificate container and chains to the pinned Cryptnox attestation root. `--csr` additionally checks that the attested key is the one in a CSR. The Cryptnox production root ships bundled, so `verify-attestation` anchors the chain out of the box. Where no anchor covers a chain it reports that the chain cannot be verified — it never passes silently.

To add anchors without rebuilding the package, point `CRYPTNOX_TRUST_DIR` at a directory of PEM certificates:

```
$ export CRYPTNOX_TRUST_DIR=/etc/cryptnox/anchors
$ cryptnox-id piv verify-attestation
```

Certificates are classified by inspection: self-issued CAs become pinned roots, other CAs become intermediates, and non-CA material is ignored. The directory is *added* to whatever ships bundled (duplicates are collapsed), so it extends the trust store rather than replacing it. `genuine verify --anchors DIR` accepts a directory the same way for that command alone.

18.7 Personalization (`piv perso`)

```
piv perso set-pin                 set the initial PIN (over SCP03)
piv perso set-puk                 set the initial PUK (over SCP03)
piv perso create-key-object       add a key object for a (slot, algorithm) pair, with a␣
↪role
piv perso generate-key            on-card key generation (per the slot's key objects)
piv perso import-key              inject an externally generated private key
piv perso import-p12              one-command PKCS#12 (.p12/.pfx) key + certificate␣
↪import
piv perso generate-csr            PKCS#10 CSR, signed on-card
piv perso self-sign-cert          self-signed certificate (dev/test), signed on-card
piv perso import-cert             write a certificate into the slot container
                                  (--csr / --verify-pin check it is for the slot's key)
piv perso generate-chuid          build a minimal CHUID object
piv perso generate-ccc            build a minimal CCC object
piv perso generate-discovery      build the Discovery object
piv perso write-object            write any data object from a file
piv perso write-standard-objects  generate + write CHUID and CCC
piv perso smoke-test              verify PIN + one on-card signature
```

18.8 Notes

- Signing needs a key whose role includes **SIGN**. On `cryptnox-default`, 9C signs and 9A (AUTHENTICATE only) returns `6985`; `generate-csr`, `self-sign-cert` and `smoke-test` then name the key's role, which `piv inventory` lists for every key object, and point at another key on the same slot that can sign when there is one.
- `piv perso create-key-object` adds a key object for one slot and algorithm, with the role (`--role SIGN+AUTHENTICATE`, say) and access modes you give (default `PIN` over contact, `NEVER` over contactless). The object is permanent: it cannot be changed or deleted, and a finalized card refuses new ones. A slot holds one object per algorithm, so a 9A that cannot sign gets a signing key from a second object for another algorithm, for example `--slot 9A --algorithm RSA2048 --role SIGN+AUTHENTICATE`; the first object stays on the card, unused. The applet refuses a key object with both `SIGN` and `KEY_ESTABLISH`, and so does the command. `--dry-run` shows the object and sends nothing. `generate-key --create-key-object` and `import-key --create-key-object` copy the role from `cryptnox-default` and name it; on 9A or 9C they ask before creating a key that cannot sign. `quickstart` copies the role and the RSA-CRT attribute from the profile it was given.
- On-card generation works for whichever **mechanisms the slot has a key object for**, which is decided by the pre-personalization profile, not by the applet. On `cryptnox-default` every key object is ECC (P-256/P-384), so RSA has to be imported there. `ms-logon` also creates an RSA-2048 object on 9A, and `generate-key --slot 9A --algorithm RSA2048` generates on-card on such a card. Asking for a mechanism the slot has no object for returns `6A80`.
- Some cards cannot return an RSA public-key template larger than 256 bytes over the admin channel. `generate-key` detects that from the card's own response and finishes the generation over plain APDUs, authenticating the PIV management key (`9B`) first. The management key is contact-only and its value comes from `--default-keys` (development cards) or `PIV_MGMT_KEY`; a card whose `9B` object holds no value is reported with the command that loads one (`factory piv preperso set-mgmt-key`). ECC is unaffected, and cards that return the full template take the admin channel as before.
- When that path is taken, the key pair from the first attempt is replaced by the one generated over the plain path, whose public half is the one reported; RSA generation therefore runs twice on such a card.
- `piv quickstart` defaults to ECC P-256, except with `--profile ms-logon` on slot 9A, the one built-in shape with an RSA key object and the Windows one, where it defaults to RSA-2048 because Windows does not enumerate EC keys. Elsewhere `--algorithm RSA*` is accepted only when the card already has that RSA key object for the slot.
- `piv quickstart` reads the applet's key-object listing before planning. A card with no structure at all needs `--include-preperso` (or a `load-config`); a card whose structure lacks the key object for the requested slot and mechanism gets that one object created with `--include-preperso`, never a second `load-config` over the existing structure, which the applet refuses with `6E27`. Before writing anything it refuses a certificate step that would end in `6985`: an existing key without the SIGN role (when the card lists its key objects), or a key object the profile would create without it. A certificate in the slot counts for the key it was issued to. When it belongs to the slot's other key and that key cannot sign, quickstart generates the requested key and replaces the certificate, and says so; any other certificate it refuses to overwrite.
- `piv perso import-cert` writes the certificate it is given. `--csr FILE` refuses one whose public key is not the CSR's, with no PIN; `--verify-pin` has the slot sign a challenge and refuses unless the certificate's key verifies it. Without either, a certificate for another key is written and fails only when used.
- `piv perso import-key` is the way to place an **externally generated** RSA key. It sends the key as CHANGE REFERENCE DATA ADMIN elements over SCP03 and starts with a CLEAR, so re-runs are safe.
- Large objects (certificates) are written with ISO command chaining automatically.

CHAPTER

NINETEEN

QUICK START: A PASSKEY ON THE CARD

Create a credential and prove the whole loop:

```
$ cryptnox-id fido info
$ cryptnox-id fido credential self-test
```

`credential self-test` registers a credential, requests an assertion, and verifies the returned signature — the full WebAuthn round trip against the real authenticator. A plain (non-resident, no user-verification) credential doesn't need a PIN set at all on this authenticator (`fido info` reports `option alwaysUv: False`).

For a **resident (discoverable), PIN-verified** credential instead:

```
$ cryptnox-id fido pin set
$ cryptnox-id fido credential self-test --rk --pin-env CRYPTNOX_FIDO_PIN
$ cryptnox-id fido credential list --pin-env CRYPTNOX_FIDO_PIN
```

`credential list` shows the credential stored and discoverable on the card. Setting the PIN is one-way: it can only be changed afterward, not removed, short of a full `fido reset` (which erases every credential).

Note

If more than one PC/SC reader is present, the CLI's default reader auto-pick can grab the wrong one — pass `--reader <substring>` explicitly (matching your reader's name) rather than assume it found the card you meant.

Note

On **Windows**, run from an **Administrator** terminal: the OS reserves the FIDO applet from non-elevated processes. macOS and Linux need no elevation. User presence is satisfied automatically over the interface this was tested on (contact) without a separate physical tap.

19.1 Credential management

```
$ cryptnox-id fido credential list --pin-env CRYPTNOX_FIDO_PIN
$ cryptnox-id --yes fido credential delete --credential-id <id> --pin-env CRYPTNOX_FIDO_
↪PIN
```

Deleting a credential drops the count reported by `list` immediately. `delete` asks for interactive confirmation; script it with the global `--yes` flag.

19.2 authenticatorConfig policy

```
$ cryptnox-id fido config show
$ cryptnox-id fido config toggle-always-uv --pin-env CRYPTNOX_FIDO_PIN
```

`config show` is read-only. `toggle-always-uv` flips `alwaysUv` each call, so running it twice restores the original state; when on, every credential operation requires user verification regardless of what the relying party asks for.

```
$ cryptnox-id fido config min-pin-length --length 5 --pin-env CRYPTNOX_FIDO_PIN
```

Unlike `toggle-always-uv`, this is one-way: it only raises the minimum, and the only way back down is a full reset.

19.3 Reset

```
$ cryptnox-id fido reset --i-understand-this-wipes-all-credentials
```

Erases every credential and the PIN, and drops the minimum PIN length back to its default. `authenticatorReset` normally has to be sent shortly after the card is presented (a CTAP2 anti-remote-reset protection); if it's refused, remove and reinsert the card and try again immediately.

CHAPTER

TWENTY

FIDO2 GUIDE

The FIDO2 applet (AID `A0000006472F0001`) is reached with CTAP messages tunnelled over ISO 7816 APDUs (`80 10 00 00 [CTAP cmd][CBOR]`). For the condensed version, see *Quick start: a passkey on the card*.

20.1 Windows: run elevated

On Windows the OS **blocks non-elevated processes** from talking to the FIDO CTAP AID over PC/SC — you get `SCARD_E_NO_ACCESS (0x80100027)`. This is enforced by Windows itself (to reserve FIDO for the WebAuthn platform API), not by the card, and it applies on **every** reader, contact or contactless.

Run `fido` commands from an **Administrator terminal**. Every `fido` command surfaces this requirement up front — a warning if the process isn't elevated (the call then fails with the error above), or a confirmation that it is. In `--json` mode the note goes to stderr so stdout stays pure JSON. `doctor` also reports the elevation state.

20.2 Read-only commands

```
$ cryptnox-id fido ping            # SELECT the applet, show its version string
$ cryptnox-id fido info            # authenticatorGetInfo (capabilities)
$ cryptnox-id fido get-info        # alias of `info`
$ cryptnox-id fido pin status      # is a clientPIN set? retries remaining?
```

`fido info` decodes the CTAP `authenticatorGetInfo` map: supported versions (`FIDO_2_0`, `FIDO_2_1`, ...), extensions, the 16-byte **AAGUID** (matched against the known Cryptnox AAGUIDs), options (`rk`, `clientPin`, `credMgmt`, ...), max message size, PIN/UV protocols, transports, algorithms, and firmware version.

`fido pin status` reads the `clientPin` option and, when a PIN is configured, queries `clientPIN.getPinRetries` — non-destructive, it doesn't consume a retry.

20.3 Write operations

These use the CTAP2 **PIN/UV Auth Protocol** (ECDH key agreement -> AES/HMAC under protocol 1 or 2). PINs are entered by masked prompt or a `--*-env` variable, never on the command line, and are registered with the redactor.

```
# PIN management
$ cryptnox-id fido pin set                      # set the INITIAL PIN (masked, confirmed)
$ cryptnox-id fido pin change                    # change an existing PIN

# Credentials (require user presence - a tap)
$ cryptnox-id fido credential create  --rp-id example.com [--rk] [--pin-env P]
$ cryptnox-id fido credential assert  --rp-id example.com [--credential-id HEX] [--pin-
```

(continues on next page)

(continued from previous page)

```
↪env P]
$ cryptnox-id fido credential self-test --rp-id example.com [--pin-env P]   # register->
↪assert->verify

# Credential management (resident credentials; PIN required)
$ cryptnox-id fido credential list   --pin-env P            # enumerate resident␣
↪credentials
$ cryptnox-id fido credential delete --credential-id HEX --pin-env P

# Destructive
$ cryptnox-id fido reset --i-understand-this-wipes-all-credentials   # + typed RESET-FIDO
```

`credential self-test` is the quickest end-to-end check: it registers a credential, gets an assertion, and verifies the ES256 signature against the registered key, reporting PASS/FAIL.

Note

A FIDO PIN cannot be removed except by `fido reset`, which erases **all** credentials and the PIN along with it. `reset` is gated by a flag and a typed `RESET-FIDO` confirmation, and — like most authenticators — only works shortly after the card is presented, with a tap; if it's refused, remove and reinsert the card and try again immediately. `--protocol 1|2` selects the PIN/UV auth protocol (1 is the universal default).

20.4 Credential management

`fido credential list` enumerates resident (discoverable) credentials — relying party, user, and credential ID for each. `fido credential delete --credential-id HEX` removes one; the count reported by a follow-up `list` drops immediately. `delete` asks for interactive confirmation; script it with the global `--yes` flag.

`fido pin change` requires the current PIN as well as the new one — a wrong current PIN decrements the retry counter, same as any other failed PIN attempt, while a successful change doesn't cost a retry.

20.5 authenticatorConfig policy

`authenticatorConfig` (CTAP `0x0D`) changes device-wide policy. This applet exposes two subcommands; each is authorized by a pinUvAuthToken carrying the **authenticatorConfiguration** permission, so pass `--pin-env NAME` when a clientPIN is set. (The pinUvAuthParam is MACed over a mandatory prefix of 32 `0xFF` bytes — per the CTAP 2.1 spec, not a card quirk.)

```
$ cryptnox-id fido config show                                  # read-only policy view
$ cryptnox-id fido config toggle-always-uv --pin-env P          # flip alwaysUv␣
↪(reversible)
$ cryptnox-id fido config min-pin-length --length 6 --pin-env P  # raise the minimum␣
↪(one-way)
```

- ``**config show**`` reports `alwaysUv`, the minimum PIN length, and which config operations the applet exposes (`authnrCfg`, `setMinPINLength`); it also flags that bio enrollment, large-blob, and enterprise attestation are **not** implemented by this applet.
- ``**toggle-always-uv**`` flips `alwaysUv`; when on, every `makeCredential`/`getAssertion` requires user verification. Running it twice restores the original state.

- ``**min-pin-length**`` raises the minimum PIN length. It is **one-way**: the value can only increase; lowering it again requires `fido reset`, which also erases every credential.

20.6 Scope

Implemented: read-only inspection, PIN set/change, `makeCredential`, `getAssertion`, `reset`, credential management (metadata, enumerate, delete), and `authenticatorConfig` (`alwaysUv` toggle + `setMinPINLength`).

Not implemented by this applet: **bio enrollment** and **large-blob** (confirmed via `getInfo`: no `bioEnroll`/`largeBlobs` option, and the card has no fingerprint sensor); **enterprise attestation** is likewise not advertised (`ep` absent).

20.7 Next steps

- *Quick start: a passkey on the card* — the condensed walkthrough this guide expands on

CHAPTER

TWENTYONE

FIDO2 COMMANDS

FIDO2 authenticator management over NFC (PC/SC). **Windows requires an Administrator terminal** — the OS reserves the FIDO applet identifier from non-elevated processes; the CLI detects this and says so. PINs come from `--pin-env` variables or masked prompts.

21.1 Inspection

```
fido ping          SELECT + version string
fido info          authenticatorGetInfo, summarized
fido get-info      authenticatorGetInfo, raw decoded map
```

21.2 PIN

```
fido pin status    PIN set? retries left? (non-destructive)
fido pin set       set the initial PIN
fido pin change    change the PIN
```

21.3 Credentials

```
fido credential create      makeCredential (WebAuthn registration)
fido credential assert      getAssertion (WebAuthn authentication)
fido credential self-test   register -> assert -> verify the signature
fido credential list        resident (discoverable) credentials
fido credential delete      delete a resident credential
```

User presence over NFC is satisfied by card presence on the reader. A usable credential requires user verification — set a PIN first.

21.4 Configuration

```
fido config show              authenticator options and policy
fido config toggle-always-uv  flip the alwaysUv policy (reversible)
fido config min-pin-length    raise the minimum PIN length (one-way)
```

21.5 Destructive

```
fido reset    authenticatorReset: wipes ALL credentials and the PIN
              (gated by --i-understand-this-wipes-all-credentials)
```

Not exposed by this card: bio enrollment, large blobs, enterprise attestation.

CHAPTER

TWENTYTWO

QUICK START: A TAMPER-EVIDENT NFC TAG (SDM/SUN)

MIFARE DESFire EV3 **Secure Dynamic Messaging** (SDM), also marketed as **Secure Unique NFC** (SUN), turns a free-read file into a self-authenticating tag: on *every* read, the card mirrors a freshly encrypted copy of its UID and an incrementing read counter into the file content, plus a MAC over them. A backend that knows the key can prove each read came from the genuine card — and detect replayed or cloned URLs — without any session or pairing.

This guide configures SDM on a Cryptnox card whose DESFire function is EV3 and verifies the whole loop with `cryptnox-id`. Cards with DESFire EV2 do not support SDM: the card refuses SDM files, and `mifare sdm` stops with a message naming the generation. It follows the public NXP application note AN12196.

22.1 Prerequisites

- A **DESFire-capable contactless reader** (the ACS ACR1252 is verified; some contactless readers do not pass native DESFire commands through).
- A Cryptnox card with the DESFire EV3 function. `mifare version` names the generation:

```
$ cryptnox-id mifare version
```

22.2 Step 1 — create an application

SDM uses two application keys besides the master key: one to encrypt the mirrored UID + counter (key 2) and one for the MAC (key 1). Create the application with at least 3 AES keys — on a fresh application all keys are all-zero:

```
$ cryptnox-id mifare app create --aid CC0102 --keys 3
```

22.3 Step 2 — set up the SDM file

One command creates the file (SDM must be enabled at file creation on EV3), writes an NDEF-style URL template with placeholders for the two mirrors, and configures the SDM options:

```
$ cryptnox-id mifare sdm setup --aid CC0102 --file-id 02 --zero-key \
    --url "https://example.com/t"
```

(`--zero-key` authenticates with the publicly known all-zero factory-default key - right for a fresh application; use `--key-env` once keys are rotated.)

The stored content becomes:

```
https://example.com/t?picc=00000000000000000000000000000000&c=0000000000000000
```

`picc=` receives the encrypted UID + read counter (32 hex characters) and `c=` the SDMMAC (16 hex characters), recomputed by the card on every read.

22.4 Step 3 — read and verify

```
$ cryptnox-id mifare sdm read --aid CC0102 --file-id 02
NDEF: https://example.com/t?picc=1F44...&c=9A02...
  decrypted UID: 046F9C6A7B7180
  read counter:  3
  SDMMAC: verified (with the all-zero factory default keys)
```

`sdm read` does exactly what a verification backend would: free-read the file, decrypt the `picc=` mirror into the card UID and read counter, derive the per-read session key, and check the `c=` MAC. Run it twice — the ciphertext changes completely, the counter increments, and the MAC stays valid. That is the SUN property: every read is a fresh, verifiable proof.

22.5 Scope and production notes

- **What this demonstrates** is the complete SDM/SUN mechanism, with `cryptnox-id` acting as both tag programmer and verification backend. Exposing the file as a *phone-tappable* NFC tag additionally requires the standard NDEF Type 4 application layout — that is an integration step outside this quick start.
- **Rotate the keys.** The demo runs with the factory all-zero application keys. For production, change keys 1 and 2 (and the application master key) first — `mifare keys change`, see *MIFARE DESFire guide* — and keep the verification keys server-side only.
- A verification backend should also check the read counter is **strictly increasing** per UID to reject replays of older URLs.

22.6 Next steps

- *MIFARE DESFire guide* — applications, files, authentication, value and record files, key rotation
- *MIFARE DESFire commands* — every `mifare` command

CHAPTER

TWENTYTHREE

MIFARE DESFIRE GUIDE

The DESFire function is the card's **default applet** — it answers a fresh contactless connection before any JavaCard applet is selected. All `mifare` commands need a **DESFire-capable contactless reader** (the ACS ACR1252 is verified; some contactless readers do not pass native DESFire commands through).

The CLI speaks authenticated DESFire (EV2 secure messaging: AES session keys, command/response MACs, optional full encryption) and the DESFire EV3 Secure Dynamic Messaging feature, implemented against public NXP documentation (application note AN12196 and the DESFire EV3 short data sheet).

23.1 Inspect (read-only)

```
$ cryptnox-id mifare info          # version + free memory + applications
$ cryptnox-id mifare version       # hardware/software version, storage, UID
$ cryptnox-id mifare free-memory
$ cryptnox-id mifare apps list
$ cryptnox-id mifare files list --aid CC0102
```

23.2 Applications and files

```
# 3 AES keys (key 0 = application master); new keys default to all-zero
$ cryptnox-id mifare app create --aid CC0102 --keys 3

# standard data file 0x01, 32 bytes, free read / keyed write
$ cryptnox-id mifare files create-standard --aid CC0102 --file-id 01 --size 32
```

23.3 Authenticate, write, read

`AuthenticateEV2First` establishes AES session keys and a transaction identifier. Writes are sent MAC-protected (command and response MACs verified by the CLI); free-read files read without authentication.

```
$ cryptnox-id mifare keys authenticate --aid CC0102 --key-no 0 --zero-key

$ cryptnox-id mifare write --aid CC0102 --file-id 01 --data CAFEBABE --zero-key
$ cryptnox-id mifare write --aid CC0102 --file-id 01 --in payload.bin --zero-key

$ cryptnox-id mifare read --aid CC0102 --file-id 01 --length 4
$ cryptnox-id mifare read --aid CC0102 --file-id 01 --out dump.bin
```

Writes larger than one native frame are split into command-chaining frames automatically — the only ceiling is the file size.

Standard data files also support **fully encrypted** transfer: create the file with `--full`, then `write --full` / `read --full --length N --zero-key` (encrypted reads need the key too).

23.4 Delete (authenticated)

Deleting an application requires authentication with the application master key; the CLI authenticates and sends a MAC-protected `DeleteApplication`:

```
$ cryptnox-id mifare app delete --aid CC0102 --zero-key
```

The card does not return the memory a deleted application used: `mifare free-memory` stays lower until the card is formatted with `mifare format` (see *Format the PICC (destructive)* below for the key it needs).

23.5 Value files

A value file holds a signed integer changed by credit/debit, persisted with a transaction commit (issued by the CLI in the same authenticated session):

```
$ cryptnox-id mifare value create --aid CC0102 --file-id 02 \
    --initial 100 --lower 0 --upper 1000000
$ cryptnox-id mifare value get    --aid CC0102 --file-id 02 --zero-key
$ cryptnox-id mifare value credit --aid CC0102 --file-id 02 --amount 50 --zero-key
$ cryptnox-id mifare value debit  --aid CC0102 --file-id 02 --amount 30 --zero-key
```

23.6 Record files

Linear or cyclic record files; write and clear commit a transaction:

```
$ cryptnox-id mifare record create --aid CC0102 --file-id 03 \
    --record-size 8 --max-records 4 [--cyclic]
$ cryptnox-id mifare record write  --aid CC0102 --file-id 03 \
    --data AABBCCDDEEFF0011 --zero-key
$ cryptnox-id mifare record read   --aid CC0102 --file-id 03 --zero-key
$ cryptnox-id mifare record clear  --aid CC0102 --file-id 03 --zero-key
```

Reading an empty record file returns `BOUNDARY_ERROR (0xBE)` — expected, it simply has no records yet.

23.7 Keys

DESFire AES keys come from `--zero-key` (the all-zero factory default of a new application) or `--key-env NAME` (hex in an environment variable) — never on the command line. Application keys are AES-128. The one 2K3DES key this tool uses is a factory PICC master key, for `mifare format` and `mifare keys picc-to-aes`.

Rotate a key (same-key change — authenticate with the key being changed):

```
$ cryptnox-id mifare keys change --aid CC0102 --key-no 0 --zero-key \
    --new-key-env NEWKEY
$ cryptnox-id mifare keys authenticate --aid CC0102 --key-no 0 --key-env NEWKEY
```

Cross-key change — authenticate with a *different* key (typically the application master, key 0) to change another key. The card requires the target key's **current value** to authorize the change — supply it via `--zero-key` / `--key-env`:

```
$ cryptnox-id mifare keys change --aid CC0102 --key-no 1 --zero-key \
    --new-key-env NEWKEY1 --auth-key-no 0 --auth-zero-key
$ cryptnox-id mifare keys authenticate --aid CC0102 --key-no 1 --key-env NEWKEY1
```

A wrong current value is cleanly rejected (`INTEGRITY_ERROR (0x1E)`) and the key is left unchanged. A lost application key is recoverable only via the PICC master key.

23.8 Secure Dynamic Messaging (SDM / SUN)

DESFire EV3 cards can mirror an encrypted UID + read counter and a MAC into a free-read file on every read: a self-authenticating NFC tag. DESFire EV2 does not support SDM, and `mifare sdm` refuses a card whose `mifare version` reports EV1 or EV2. The *Quick start: a tamper-evident NFC tag (SDM/SUN)* walks through it end to end; the short form:

```
$ cryptnox-id mifare sdm setup --aid CC0102 --file-id 02 --zero-key \
    --url "https://example.com/t"
$ cryptnox-id mifare sdm read  --aid CC0102 --file-id 02
```

`sdm setup` creates the file with the SDM option enabled (an EV3 file-creation property), writes the URL template, and configures UID + read-counter mirroring with the application's key 2 encrypting the mirrored data and key 1 keyed into the MAC. `sdm read` plays verification backend: it decrypts the mirror, checks the MAC, and reports the UID and read counter. Configuration and verification follow the public NXP application note AN12196.

For production, rotate keys 1 and 2 away from the factory zero key first and keep them server-side; verify reads by checking the MAC **and** a strictly increasing counter per UID.

23.9 Format the PICC (destructive)

`mifare format` erases **every** application and file on the card via `FormatPICC`; the PICC master key and its settings survive. It is gated by `--i-understand-this-erases-all-applications` (or a typed confirmation) and authenticates with the PICC master key:

```
$ cryptnox-id mifare format --zero-key --i-understand-this-erases-all-applications
```

`FormatPICC` authenticates with the **PICC master key**. On factory cards that key is 2K3DES (all zero); `format` reads the key type and authenticates accordingly, so `--zero-key` works on a factory card and on one whose PICC master key is the all-zero AES key. A 3K3DES PICC master key is not supported.

To move the PICC master key to AES-128, so the card's other AES commands can use it:

```
$ cryptnox-id mifare keys picc-to-aes --zero-key --new-key-env NEW_PICC_KEY
```

`--new-key-env` names an environment variable holding the new key in hex. The command authenticates with the new key afterwards to confirm it. This tool has no way back to 2K3DES.

23.10 Scope

Covered: read-only inspection; standard data files (plain, MAC-protected and fully encrypted transfer, chunked writes); value files; record files (linear/cyclic) — all with transaction commit; same-key and cross-key `ChangeKey`; SDM/SUN configuration and host-side verification; `FormatPICC` with an AES or 2K3DES PICC master key, and moving a 2K3DES PICC master key to AES. Other DES/3DES keys are not supported.

CHAPTER

TWENTYFOUR

MIFARE DESFIRE COMMANDS

Contactless MIFARE DESFire commands (DESFire-capable reader required). Keys are AES-128, except a factory 2K3DES PICC master key for `format` and `keys picc-to-aes`, supplied via `--zero-key` (factory default) or `--key-env NAME`, never on the command line.

24.1 Inspection

```
mifare info           version + free memory + applications
mifare version        hardware/software version, storage size, UID
mifare free-memory    free EEPROM
mifare apps list      application directory
```

24.2 Applications

```
mifare app create     create an application (AES keys; new keys all-zero)
mifare app delete     delete an application and every file in it
                      (authenticated; PERMANENT - see Destructive below)
```

24.3 Keys

```
mifare keys authenticate   prove a key / session crypto (AuthenticateEV2First)
mifare keys change         rotate a key (same-key or cross-key)
```

24.4 Files and data

```
mifare files list             file directory of an application
mifare files create-standard  create a standard data file (plain/MAC/FULL)
mifare write                  write data (MAC-protected; --full for encrypted)
mifare read                   read data (plain for free-read; --full for encrypted)
```

24.5 Value and record files

```
mifare value create    create a value file (bounded signed counter)
mifare value get       read the current value
mifare value credit    add to the value (commits the transaction)
```

(continues on next page)

(continued from previous page)

```
mifare value debit     subtract from the value (commits the transaction)

mifare record create   create a linear or cyclic record file
mifare record write    append a record (commits)
mifare record read     read records
mifare record clear    clear the record file (commits)
```

24.6 Secure Dynamic Messaging (EV3)

```
mifare sdm setup     create + configure an SDM/SUN file with a URL template
mifare sdm read      read the file, decrypt the PICC mirror, verify the MAC
```

See the *Quick start: a tamper-evident NFC tag (SDM/SUN)* for the end-to-end flow.

24.7 Destructive

```
mifare app delete    delete an application and every file in it, permanently;
                     the memory is freed only by mifare format
                     (confirmation required unless --yes)
mifare format        FormatPICC: erase ALL applications (gated; the PICC
                     master key, AES or 2K3DES)
mifare keys picc-to-aes
                     replace a 2K3DES PICC master key with an AES-128 key
                     (confirmation required unless --yes; no way back)
```

CHAPTER

TWENTYFIVE

QUICK START: PROVE A CARD IS GENUINE

On a **contact** reader (the genuineness applet does not answer contactless). Production cards with FIDO2 do not carry the applet; see *Genuineness guide* for what shows those cards are genuine.

```
$ cryptnox-id genuine info
$ cryptnox-id genuine verify
```

`genuine info` reports whether the attestation applet is present and personalized, and shows the on-card device certificate's subject, issuer and serial — no verification yet.

`genuine verify` runs the two checks that prove genuineness:

1. **proof of possession** — the card signs a fresh host nonce with its on-card device key, so a copied certificate cannot pass;
2. **certificate chain** — the on-card certificate is chained to the **pinned** Cryptnox root.

The verdict is **GENUINE** only when *both* pass. The Cryptnox production root ships bundled with the tool, so a production card anchors and verifies out of the box:

```
$ cryptnox-id genuine verify
Genuineness: GENUINE
  proof of possession (ATTEST): valid
  certificate chain: verified
```

Note

A **development** card chains to a throwaway dev root and can never pass as genuine Cryptnox hardware — its verdict stays **NOT proven**. To test the flow against a dev PKI, anchor it explicitly:

```
$ cryptnox-id genuine verify --anchors <dev-ca-dir>
```

25.1 Next steps

- *Genuineness guide* — what is checked, what deliberately is not, and how trust anchors work
- *Genuineness commands* — every `genuine` command

CHAPTER

TWENTYSIX

GENUINENESS GUIDE

The Cryptnox **genuineness / attestation applet** (RID `A0000010`, AID `A000001000024701`) carries an on-card EC device key and a device certificate that chains to the Cryptnox PKI. It lets a host prove a card is genuine Cryptnox hardware without any shared secret. For the condensed version, see *Quick start: prove a card is genuine*.

The applet is **contact-only** — it does not answer over a contactless reader (SELECT returns `6A82` there) — and all `genuine` commands are **read-only**: the applet is inspected, never provisioned, by this tool.

Note

Production cards with FIDO2 do not carry the genuineness applet, and `genuine` reports it as not on the card. On those cards the FIDO2 attestation shows that the card is genuine: the attestation certificate a relying party receives at registration chains to the root that the FIDO Alliance Metadata Service lists for the card's AAGUID (`1d1b4e33-76a1-47fb-97a0-14b10d0933f1`). `fido info` shows whether the card reports a Cryptnox AAGUID; `cryptnox-id` does not verify the attestation chain.

26.1 What is checked — and what deliberately is not

There is more than one way to test a Cryptnox card's genuineness; the methods differ in what secret the verifier needs. This tool performs the two checks that need **no secret key material**:

- **Proof of possession (live).** The host sends a fresh random nonce and the card signs it with its device key (`ATTEST`); the signature is verified against the on-card leaf certificate's public key. Because the nonce is random per run, a copied certificate or a replayed signature cannot pass — the card must physically hold the device private key *now*.
- **Certificate chain.** `leaf -> Genuineness CA -> ... -> pinned Cryptnox root`, validated (name chaining, signatures, validity windows, CA constraints) and anchored on the pinned trust store. A root is trusted only because it is pinned — never because it appears in a chain read off the card.

The verdict is **GENUINE only when both pass**; proof of possession alone yields **NOT proven**.

It deliberately does **not** attempt the ISD or PIV-SSD key checks: those verify per-card, HSM/KDF-derived secret keys that this tool has no access to. Their absence is stated in the output, never hidden.

26.2 Trust anchors

The Cryptnox production root ships bundled, together with the genuineness CA that sits under it, so production cards anchor without extra setup. Where no pinned root covers the chain, the chain step reports "cannot verify" and the verdict is **NOT proven** even though proof of possession passes — the tool never silently treats an unanchored chain as genuine.

Add anchors with `--anchors DIR` (a directory of CA PEMs) or the `CRYPTNOX_TRUST_DIR` environment variable; both **extend** the bundled set rather than replacing it.

26.3 Development cards

A **dev** card chains to a throwaway dev root and can never pass as genuine Cryptnox hardware; `genuine verify --anchors <dev-ca-dir>` verifies it against the dev PKI for testing the flow only.

26.4 Inspection without the proof

`info`, `doctor` and the `report` commands surface the applet's state (present / personalized / not present) without running the proof; `report genuine` emits it as a secret-safe JSON snapshot (see *JSON output*). Only `genuine verify` produces a verdict.

26.5 Next steps

- *Quick start: prove a card is genuine* — the condensed walkthrough
- *Genuineness commands* — every `genuine` command

CHAPTER

TWENTYSEVEN

GENUINENESS COMMANDS

All `genuine` commands are **read-only** and **contact-only** — the attestation applet does not answer over a contactless reader (SELECT returns 6A82 there). Production cards with FIDO2 do not carry the applet at all, and `genuine` says so on a contact reader. What is checked, what deliberately is not, and how trust anchors work are covered in *Genuineness guide*.

27.1 Inspection

```
genuine info      SELECT + GET INFO + GET CERT; reports present / personalized,
                  device leaf subject, issuer, serial (no verification)
```

27.2 Verification

```
genuine verify    live ATTEST (proof of possession) + certificate chain to a
                  pinned Cryptnox root; verdict GENUINE only when BOTH pass

Options:
  --nonce HEX     challenge to sign (default: 32 random bytes)
  --anchors DIR   directory of extra pinned CA PEMs (e.g. a dev genuineness
                  CA) to anchor the chain
```

The Cryptnox production root ships bundled, together with the genuineness CA that sits under it, so production cards anchor without extra setup. Where no pinned root covers the chain, the verdict is **NOT proven** — extend the anchors with `--anchors` or `$CRYPTNOX_TRUST_DIR`. A **dev** card chains to a throwaway dev root and can never pass as genuine Cryptnox hardware; `genuine verify --anchors <dev-ca-dir>` verifies it against the dev PKI for testing.

CHAPTER

TWENTYEIGHT

REMOTE SERVICE GUIDE

Some PIV lifecycle operations need keys that never leave Cryptnox: the key that makes a card genuine, and the certificate authority that attests keys generated on it. The remote PIV service holds those keys in a hardware security module and performs the operation over a relayed command channel, with the card in the holder's own reader.

28.1 How a relayed operation works

1. `cryptnox-id` opens the local reader and reads the card's ATR and CPLC serial.
2. It connects to the service over TLS and names the operation.
3. The service issues a small proof-of-work; `cryptnox-id` answers it. This is an anti-abuse throttle, not a credential.
4. The service sends card commands one at a time. `cryptnox-id` checks each one against the relay policy, transmits it, and returns the card's answer verbatim, status word included. `cryptnox-id` never sends a command of its own and never reassembles a chained response; the service sees the card exactly as a local reader would.
5. The service returns a result. `cryptnox-id` shows it as the service's assertion and adds what it verified locally.

28.2 The relay policy

The card carries more than the PIV function. A relay that executed whatever it was sent would expose the wallet, the FIDO2 passkeys and the genuineness key to the service. The relay policy is an allow-list, per operation, over what a command header reveals: the selected applet, the class byte, the instruction, and for a SELECT the target application. Anything not listed is refused, the operation stops, and the refusal names the command header.

The policy also caps the number of commands and the time an operation may take, limits secure-channel authentication attempts per security domain, and never relays a card-holder PIN or PUK command. Inside a secure channel only the class and instruction bytes stay in the clear, so the policy fences which security domain the channel was opened to and which instructions pass through it; it cannot see what an encrypted command carries.

28.3 Operations

`authenticate`
: Reads the ATR and CPLC serial. Key-free. Allows only card-manager reads.

`inspect`
: Probes whether the PIV applet and its security domain are present and which key versions they carry. Key-free: the service reads each security domain's key information without authenticating, and the relay refuses every EXTERNAL AUTHENTICATE. A run leaves no failed authentication on the card.

`attest`
: Generates a key in one slot and returns a key-attestation certificate for it, signed by the Cryptnox key-attestation CA and stored in the card's attestation container as well. The relay policy admits the applet's admin channel for the key-object setup, the management-key handshake (never a slot key), key generation for the requested slot only, and the certificate write. The tool then verifies the certificate against the pinned anchors and binds it to the card, the slot and the certificate stored on the card. The certificate write itself is not bound to the slot: PUT DATA names its object inside the data, so the service could write any PIV data object during the operation, and only the certificate read back is checked. The generation itself happening on the card is the service's claim; nothing the relay sees authenticates it.

`reset`
: Wipes and reinstalls the PIV function. The service authenticates to the card manager, deletes the PIV applet, its package and its security domain, loads the applet again, recreates the security domain and loads the card's keys, then verifies the result. Every PIV key and certificate is destroyed; the keys were never exportable, so nothing can bring them back. Interactive use requires typing `RESET-PIV`; non-interactive use requires `--i-understand-this-is-irreversible`. `--yes` is not accepted.

  The relay policy for this operation admits card content management through the card manager only, lets a DELETE name nothing but the PIV instance, its package and its security domain, refuses a DELETE it cannot read unless the channel encrypts commands, caps authentication attempts per security domain, and stops at the first failed authentication so no further attempt adds to the card's count. Inside an encrypted channel the policy sees instructions, not targets; that limit is real and is why the FIDO2 and genuineness state is read before and after.

`dev-reset`
: As `reset`, for development cards, leaving the card manager and the PIV security domain on the publicly documented GlobalPlatform key so the PIV function can be pre-personalized and personalized locally. Before anything is deleted, the card must prove it is genuine with one of its own keys (see below); a card that cannot is refused untouched. Afterwards anyone with a reader can install or delete applets on the card, and only Cryptnox can make it genuine again.

28.4 What a failed authentication costs

A security domain counts every INITIALIZE UPDATE that is not followed by a successful EXTERNAL AUTHENTICATE, including one for a key version it does not hold. After five, it answers each further INITIALIZE UPDATE about 24 seconds late, until a successful authentication to it clears the count. There is no lockout, only the delay. Which security domains keep this count depends on the chip: on some only the card manager does, on others the PIV security domain does as well. This is why the relay caps authentication attempts per operation. For the PIV security domain, `piv admin authenticate` with the card's own keys clears the count; the card manager's clears only with an authentication to the card manager.

When an operation leaves INITIALIZE UPDATEs without a completed authentication, `cryptnox-id` says so afterwards with a count per security domain, also when the operation failed. With `--json` the counts are in `relay.open_initialize_updates`.

28.5 The `--default-keys` flag

By default the service authenticates to the card manager with the card's own key, derived per card from its identity. That is the state a card leaves the factory in, and a `reset` keeps it.

On `reset`, `--default-keys` asks the service to authenticate with the published GlobalPlatform default key instead and make the card genuine. That is a Cryptnox operation: the service refuses it without a Cryptnox factory token, and the refusal leaves the card untouched. A card that ended up on the default key, for example after a `dev-reset`, is re-provisioned by Cryptnox support.

On `dev-reset`, the flag selects how the card proves it is genuine before anything is deleted: without it, with its card-manager key; with it, with key version 2 of its PIV security domain. A card that can prove neither is refused untouched; Cryptnox support re-provisions it. The service's refusal may suggest `reset --default-keys`, which it refuses without a factory token.

28.6 What is sent to the service

The card's ATR and CPLC serial identify the card to the service. They are the same values any reader can obtain from the card without a key. Nothing else about the card holder is sent.

28.7 Transcripts

With `--apdu-log` or `--verbose`, relayed commands are written with their data masked unless the instruction is a known-harmless read, because the service chose the command and `cryptnox-id` cannot vouch for what it carries. `remote --full-transcript` lifts that extra masking for troubleshooting; the standard masking of secret-bearing commands still applies.

Relayed lines start with `relay`, so they read apart from the commands `cryptnox-id` sends on its own before and after the operation:

```
> 00A4040008A000000151000000000
< 6F108408A000000151000000A5049F6501FF9000
relay > 80CA9F7F00
relay < 9F7F2A...9000
```

CHAPTER

TWENTYNINE

REMOTE SERVICE COMMANDS

The `remote` commands run operations through the Cryptnox remote PIV service. The card stays in the local reader; `cryptnox-id` relays the commands the service sends to the card and reports the outcome. What the service asserts and what `cryptnox-id` verified locally are shown apart. The service, the relay policy and what each operation does to the card are described in *Remote service guide*.

Every command opens the local reader first, then connects to the service over TLS and answers a small proof-of-work before any command is relayed. Only the PIV function and its security domains are reachable through the relay; the other card functions are refused by the relay policy.

29.1 Group options

```
remote [--full-transcript] COMMAND

Options:
  --full-transcript   write relayed commands to --apdu-log and --verbose
                      without the extra masking the relay applies by
                      default; the standard secret masking still applies
```

29.2 Identification and inspection

```
remote authenticate   identify the card to the service: ATR and CPLC UID,
                      nothing else; key-free on both sides

remote inspect        probe the PIV function through the service: whether
                      the PIV applet and its security domain are present
                      and which key versions they carry; key-free but not
                      free: each key version enumerated costs one
                      INITIALIZE UPDATE the card counts as a failed
                      authentication (see the guide)
```

Both commands report the card as read locally (reader, ATR, CPLC UID), the service's result as the service asserted it, and a cross-check of the fields the two have in common.

29.3 Key attestation

```
remote attest [--slot 9C] [--algorithm ECCP256] [--ca-obj HEX] [--out FILE]

                      generate a key in the slot through the service and
```

(continues on next page)

(continued from previous page)

```
                      receive its key-attestation certificate; the service
                      also stores the certificate in the card's attestation
                      container. The slot's previous key and the container's
                      previous certificate are replaced: when either holds
                      something, the command asks, or needs --yes.

Options:
  --slot HEX          9A, 9C, 9D, 9E or a retired slot 82 to 95 (default 9C)
  --algorithm NAME    ECCP256 (default), ECCP384, RSA2048, RSA3072, RSA4096
  --ca-obj HEX        signing sub-CA in the service; omit for the
                      key-attestation CA
  --out FILE          write the returned certificate (PEM)
```

The returned certificate is verified locally before success is reported: the chain must end at a pinned trust anchor, the subject serial number must be the card's CPLC UID, the `cryptnoxPivSlot` extension must name the requested slot, the trust model must be the server-driven value, and the certified public key must equal the certificate read back from the card's attestation container. That the key was generated on the card is the service's claim; the tool cannot witness it and says so.

29.4 Reset

```
remote reset [--default-keys] [--i-understand-this-is-irreversible]

                      IRREVERSIBLY wipe and reinstall the PIV function: the
                      service deletes the PIV applet, its package and its
                      security domain, reinstalls the applet, recreates the
                      security domain and loads the card's keys. Every PIV
                      key and certificate is destroyed. Interactively the
                      phrase RESET-PIV must be typed; non-interactively the
                      flag is required. --yes is not accepted.

remote dev-reset [--default-keys] [--i-understand-this-is-irreversible]

                      Development cards only. As reset, but the card manager
                      and the PIV security domain are left on the public
                      default key, so the PIV function can be worked on
                      locally. Before anything is deleted the card must
                      prove it is genuine with one of its own keys; a card
                      that cannot is refused untouched and must be
                      re-provisioned by Cryptnox support. Only Cryptnox can
                      make the card genuine again. The phrase is
                      DEV-RESET-PIV.

Options:
  --default-keys      on reset: reserved to Cryptnox. The service would
                      authenticate with the publicly known GlobalPlatform
                      default key and make the card genuine; it refuses
                      without a Cryptnox factory token and leaves the card
                      untouched.
                      On dev-reset: the card proves it is genuine with key
                      version 2 of its PIV security domain instead of its
```

(continues on next page)

(continued from previous page)

```
                        card-manager key.
```

Before asking for consent, both commands read and show what the card holds: the PIV personalization state and data objects, the PIV security domain and its key versions, and the state of the FIDO2 and genuineness functions, which stay outside the operation. The same inventory is read again afterwards and shown next to the service's result.

29.5 Endpoint

The service endpoint is fixed to `wss://piv.cryptnox.com`. For development against a local stand-in, `CRYPTNOX_REMOTE_URL` redirects the identification and inspection commands only; it must be a `wss://` URL, or `ws://` to loopback. `reset` and `dev-reset` refuse to run while it is set.